



پرویس فنی و مهندسی شهید عباسپور



به نام خدا

تمرین سری دوم

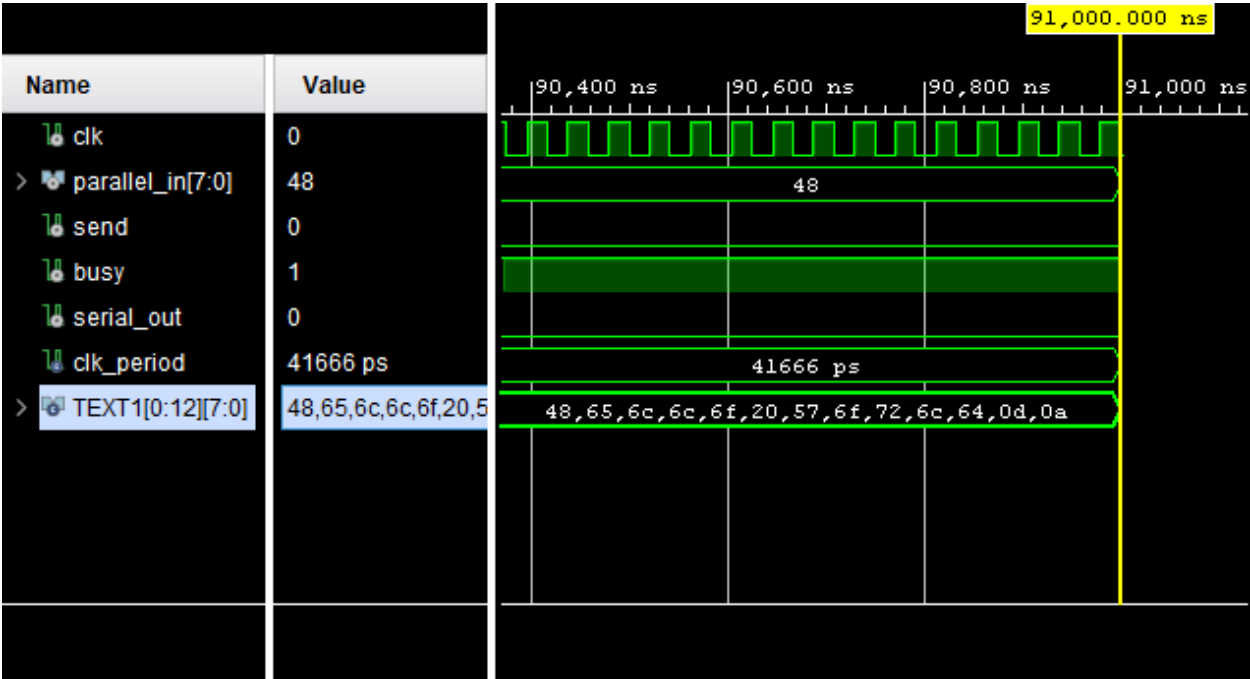
طراحی در سطح سیستم

احمد صمدپور

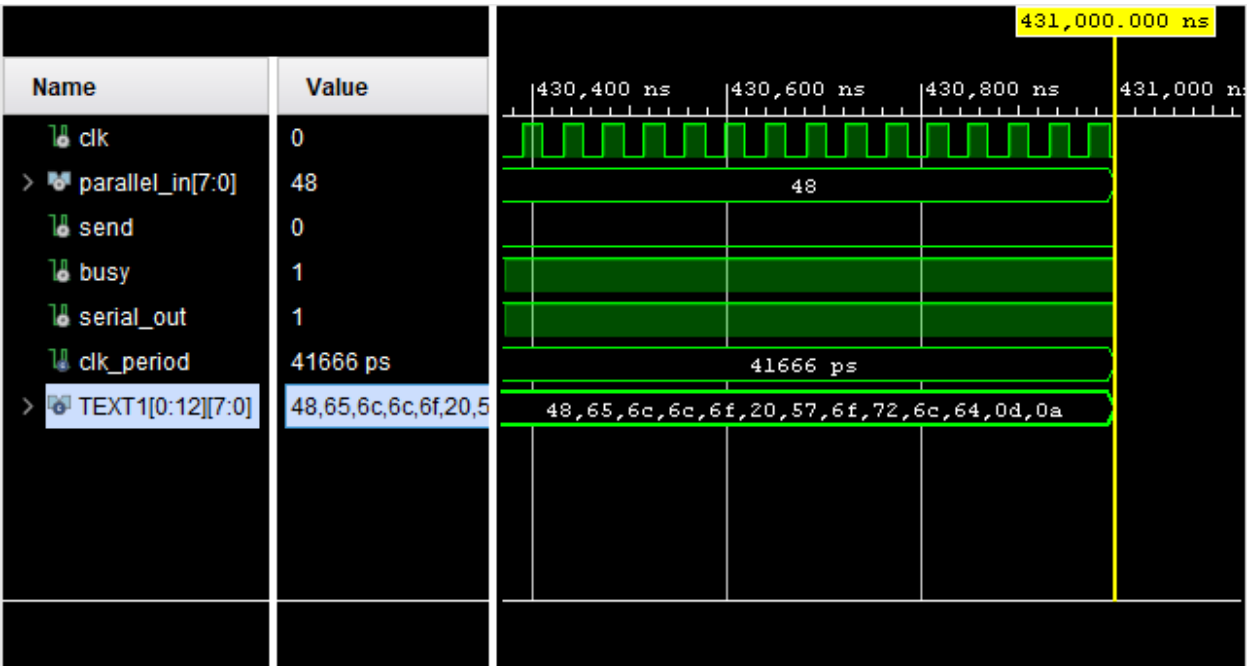
401249039

UART SEND FORM

SERIAL OUT =0;



SERIAL OUT =1



UART SEND;

```
entity UART_SEND is
  Port (
    clock : in std_logic;
    parallel_in : in std_logic_vector( 7 downto 0);
    send      : in std_logic;
    busy      : out std_logic;
    serial_out : out std_logic );
end UART_SEND;

architecture Behavioral of UART_SEND is
  signal send_packet : std_logic_vector( 9 downto 0);
  signal busy_int : std_logic := '0' ;
  signal baud_rate_count : unsigned( 11 downto 0) := (others => '0');
  signal bit_count : unsigned( 3 downto 0) :=(others =>'0');

begin

  busy <= busy_int ;

  process (clock)
  begin
    if rising_edge(clock) then
      if busy_int = '0' then
        send_packet <= '1' & Parallel_in & '0';
        if send = '1' then
          busy_int <= '1';
          Serial_out <= send_packet(0);
          bit_count <= x"1";
        end if;
      else
        baud_rate_count <= baud_rate_count + 1;
        if baud_rate_count = to_unsigned(2499, 12) then
          baud_rate_count <= (others => '0');
          if bit_count = to_unsigned(10, 4) then
            bit_count <= (others =>'0');
            Serial_out <= '1'; -- idle
            busy_int <= '0';
          else
            bit_count <= bit_count + 1;
            serial_out <= send_packet(to_integer(bit_count));
          end if;
        end if;
      end if;
    end if;
  end process;
```

UART TEST BENCH;

```
entity TB_UART_SEND is
end TB_UART_SEND;

architecture Behavioral of TB_UART_SEND is
    constant clk_period : time := 41.666 ns; -- 24MHz

    signal clk          : std_logic := '0';
    signal parallel_in  : std_logic_vector(7 downto 0) := (others => '0');
    signal send         : std_logic := '0';
    signal busy         : std_logic;
    signal serial_out   : std_logic;

    type byte_array is array (natural range <>) of std_logic_vector(7 downto 0);

    constant TEXT1 : byte_array := (
        x"48", x"65", x"6C", x"6C", x"6F", -- Hello
        x"20", -- space
        x"57", x"6F", x"72", x"6C", x"64", -- World
        x"0D", x"0A" -- CRLF
    );
begin

    -- clock
    clk_process : process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;

    -- Instantiate your component
    DUT : entity work.UART_SEND
        port map(
            clock      => clk,
            parallel_in => parallel_in,
            send       => send,
            busy       => busy,
            serial_out  => serial_out
        );

    -- stimulus
    stimulus : process
    begin
        wait for 200 ns;

        -- send twice: Hello World CRLF then Hello World CRLF
        for rep in 1 to 2 loop
```

```

-- stimulus
stimulus : process
begin
    wait for 200 ns;

    -- send twice: Hello World CRLF then Hello World CRLF
    for rep in 1 to 2 loop
        for i in TEXT1'range loop
            parallel_in <= TEXT1(i);
            send <= '1';
            wait for clk_period;
            send <= '0';

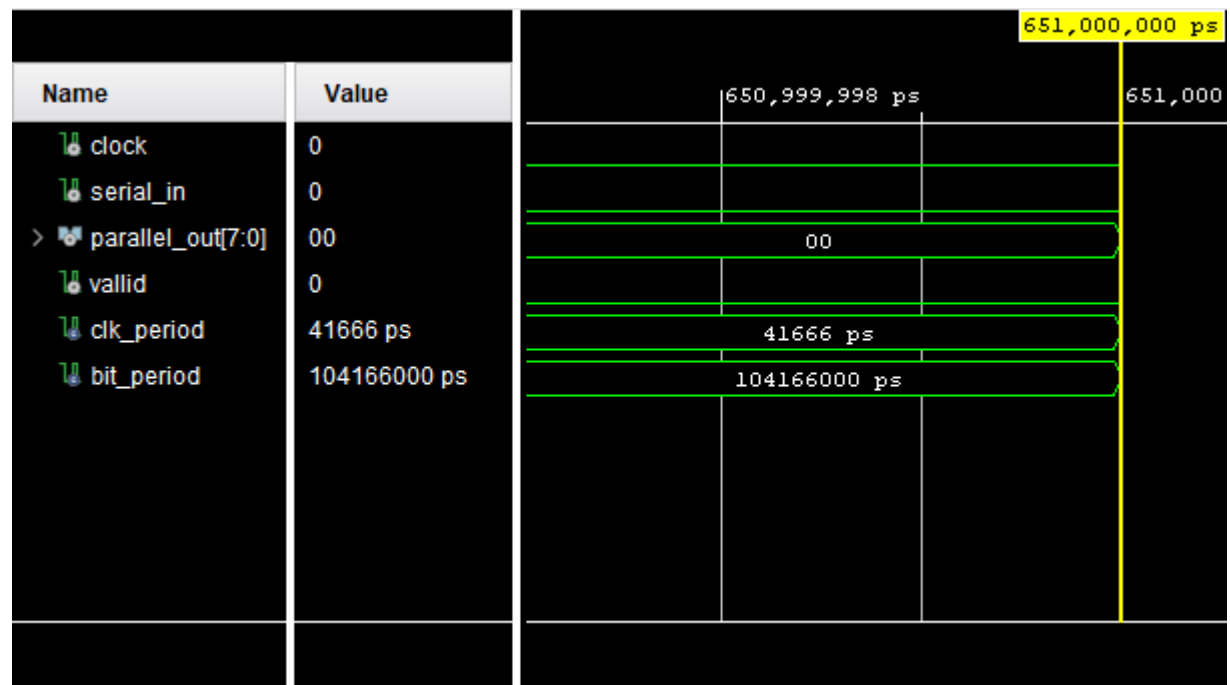
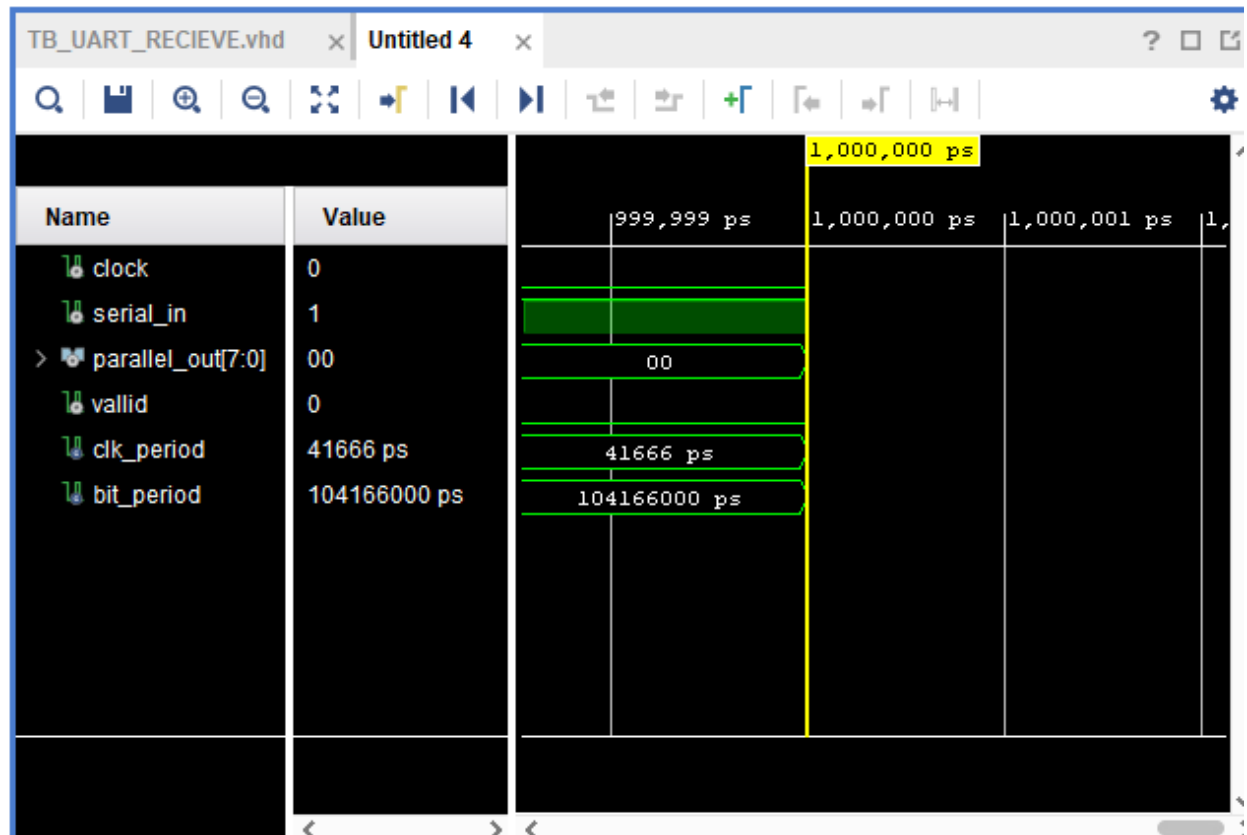
            -- wait until transmitter finishes sending
            wait until busy = '0';
        end loop;
    end loop;

    wait for 2000 ns;
    assert false report "Simulation completed." severity failure;
end process;

end Behavioral;

```

UART_RECIEVER



UART RECIEVER

Name		Value	1,051,000,000 ps	
			1,050,999,998 ps	1,051,000,000 ps
	clock	0		
	serial_in	1		
> 	parallel_out[7:0]	00	00	
	vallid	0		
	clk_period	41666 ps	41666 ps	
	bit_period	104166000 ps	104166000 ps	

(+F) itecture Behavioral of TB_UART_RECIEVE is

```
constant clk_period : time := 41.666 ns;    -- 24 MHz
constant bit_period : time := 104.166 us;    -- 9600 baud
```

```
signal clock      : std_logic := '0';
signal serial_in  : std_logic := '1';
signal parallel_out : std_logic_vector(7 downto 0);
signal vallid     : std_logic;
```

begin

```
DUT: entity work.UART_RECEIVE
  port map (
    clock      => clock,
    serial_in  => serial_in,
    parallel_out => parallel_out,
    vallid     => vallid
  );
```

```
clk_process: process
begin
  loop
    clock <= '0';
    wait for clk_period / 2;
    clock <= '1';
```

```
clk_process: process
begin
    loop
        clock <= '0';
        wait for clk_period / 2;
        clock <= '1';
        wait for clk_period / 2;
    end loop;
end process;

stim_proc: process

    procedure send_uart_byte(
        signal tx : out std_logic;
        data      : in std_logic_vector(7 downto 0)
    ) is
    begin
        -- idle
        tx <= '1';
        wait for bit_period;

        -- start bit
        tx <= '0';
        wait for bit_period;
```



```

-- data bits, LSB first
for i in 0 to 7 loop
    tx <= data(i);
    wait for bit_period;
end loop;

-- stop bit
tx <= '1';
wait for bit_period;
end procedure;

begin
    serial_in <= '1';
    wait for 500 us;

    send_uart_byte(serial_in, x"48"); -- H
    wait for 300 us;

    send_uart_byte(serial_in, x"65"); -- e
    wait for 300 us;

    send_uart_byte(serial_in, x"6C"); -- l
    wait for 300 us;

    send_uart_byte(serial_in, x"6C"); -- l
    wait for 300 us;
    send_uart_byte(serial_in, x"48"); -- H
    wait for 300 us;

    send_uart_byte(serial_in, x"65"); -- e
    wait for 300 us;

    send_uart_byte(serial_in, x"6C"); -- l
    wait for 300 us;

    send_uart_byte(serial_in, x"6C"); -- l
    wait for 300 us;

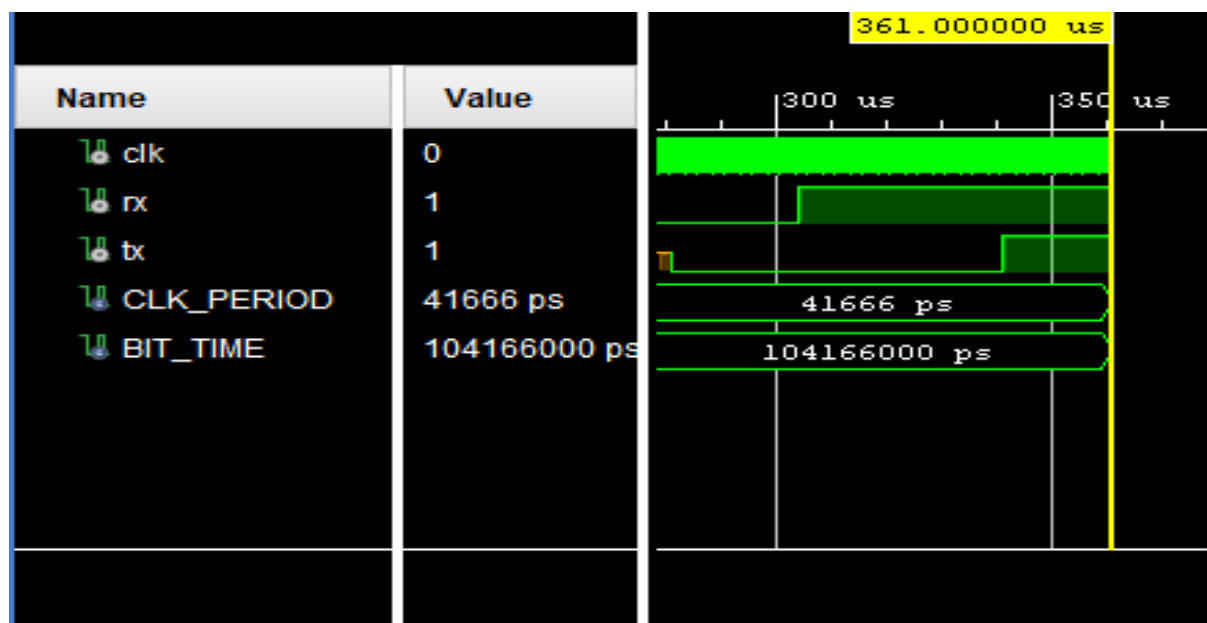
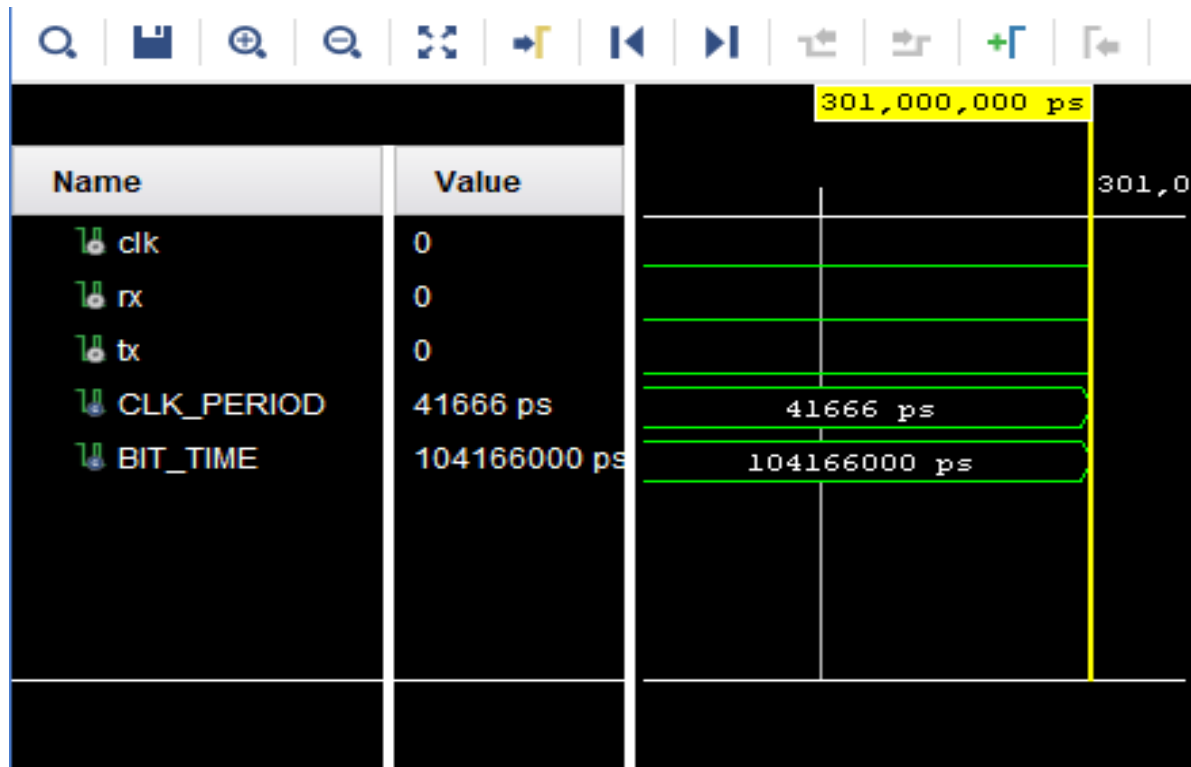
    send_uart_byte(serial_in, x"6F"); -- o
    wait for 300 us;

    wait;
end process;

end Behavioral;

```

TB ECHO:



Name		Value
clk		0
rx		1
tx		1
CLK_PERIOD		41666 ps
BIT_TIME		104166000 ps

361,000,000 ps		361,0
	41666 ps	
	104166000 ps	



```

entity TB_ECHO is
-- Port ( );
end TB_ECHO;

architecture Behavioral of TB_ECHO is

    constant CLK_PERIOD : time := 41.666 ns;    -- 24 MHz
    constant BIT_TIME    : time := 104.166 us;  -- 9600 baud

    signal clk : std_logic := '0';
    signal rx  : std_logic := '1';
    signal tx  : std_logic ;

    component ECHO
    port (
        clock      : in  std_logic;
        serial_in   : in  std_logic;
        serial_out  : out std_logic
    );
    end component;

begin

    tx <= rx;

    uut: ECHO port map (
        clock      => clk,

```

```

uut: ECHO port map (
    clock      => clk,
    serial_in  => rx,
    serial_out => tx
);
tx <= rx;
-- Clock generation
clk_process : process
begin
    while true loop
        clk <= '0';
        wait for CLK_PERIOD / 2;
        clk <= '1';
        wait for CLK_PERIOD / 2;
    end loop;
end process;

-- Stimulus
stim_process : process
begin
    -- UART line idle
    rx <= '1';
    wait for 200 us;

    -- Send 0x55 = 01010101 (LSB first)
    rx <= '0'; wait for BIT_TIME; -- start bit
    rx <= '1'; wait for BIT_TIME; -- bit0
    rx <= '0'; wait for BIT_TIME; -- bit1

    wait for 200 us;

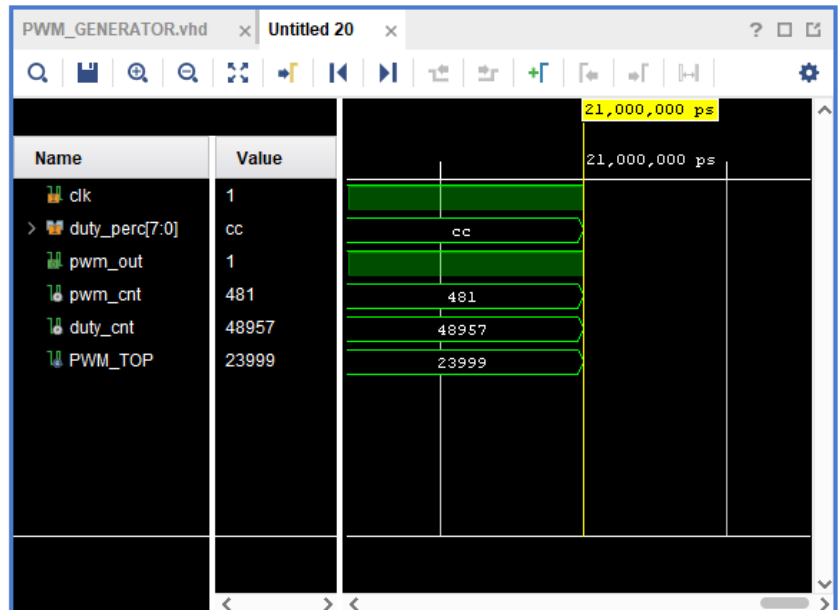
    -- Send 0x55 = 01010101 (LSB first)
    rx <= '0'; wait for BIT_TIME; -- start bit
    rx <= '1'; wait for BIT_TIME; -- bit0
    rx <= '0'; wait for BIT_TIME; -- bit1
    rx <= '1'; wait for BIT_TIME; -- bit2
    rx <= '0'; wait for BIT_TIME; -- bit3
    rx <= '1'; wait for BIT_TIME; -- bit4
    rx <= '0'; wait for BIT_TIME; -- bit5
    rx <= '1'; wait for BIT_TIME; -- bit6
    rx <= '0'; wait for BIT_TIME; -- bit7
    rx <= '1'; wait for BIT_TIME; -- stop bit

end process;
end Behavioral;

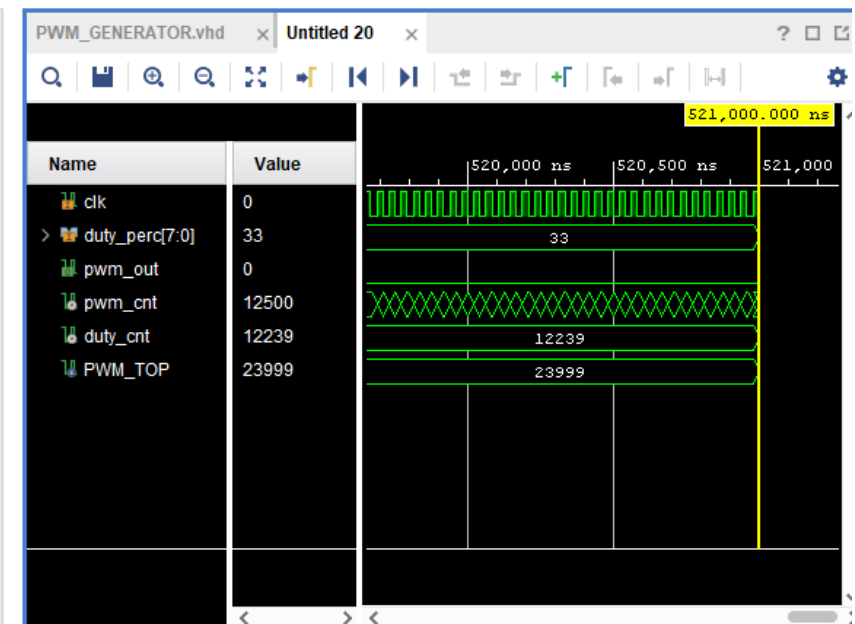
```

Pwm generator:

Objects		
Name	Value	Data Type
clk	1	Logic
> dut...	cc	Array
pw...	1	Logic
pw...	481	Integer
dut...	48957	Integer
P...	23999	Integer



Objects		
Name	Value	Data Type
clk	0	Logic
> dut...	33	Array
pw...	0	Logic
pw...	12500	Integer
dut...	12239	Integer
P...	23999	Integer



```

entity PWM_GENERATOR is
    Port ( clk : in STD_LOGIC;
          duty_perc : in STD_LOGIC_VECTOR (7 downto 0);
          pwm_out : out STD_LOGIC);
end PWM_GENERATOR;

architecture Behavioral of PWM_GENERATOR is

    constant PWM_TOP : integer := 23999; -- 24MHz / 1kHz - 1
    signal pwm_cnt : integer range 0 to PWM_TOP := 0;
    signal duty_cnt : integer range 0 to PWM_TOP := 0;

begin

    process(clk)
    begin
        if rising_edge(clk) then
            if pwm_cnt = PWM_TOP then
                pwm_cnt <= 0;
            else
                pwm_cnt <= pwm_cnt + 1;
            end if;

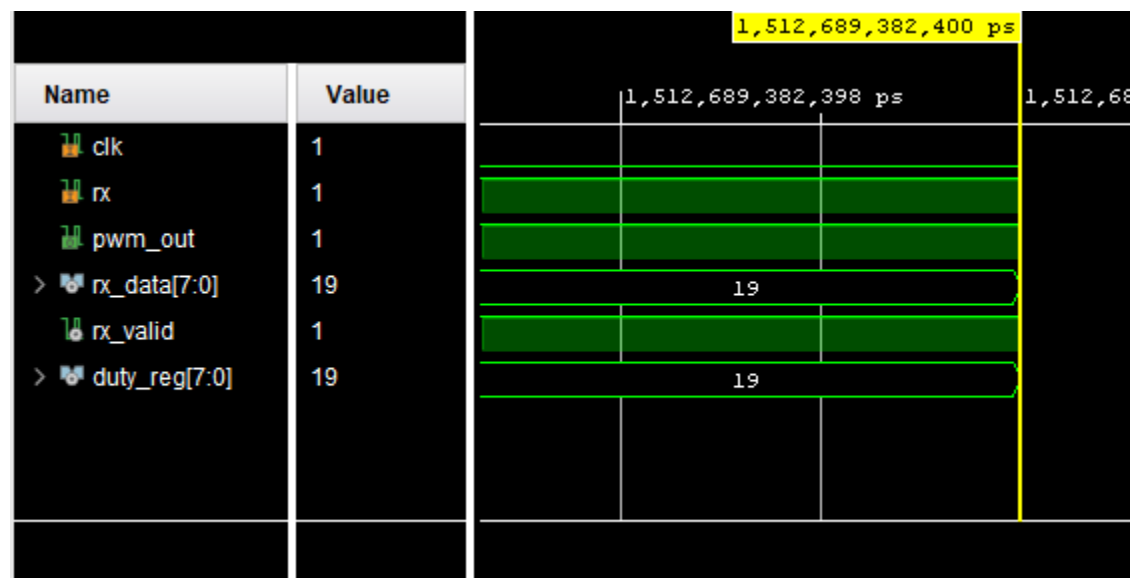
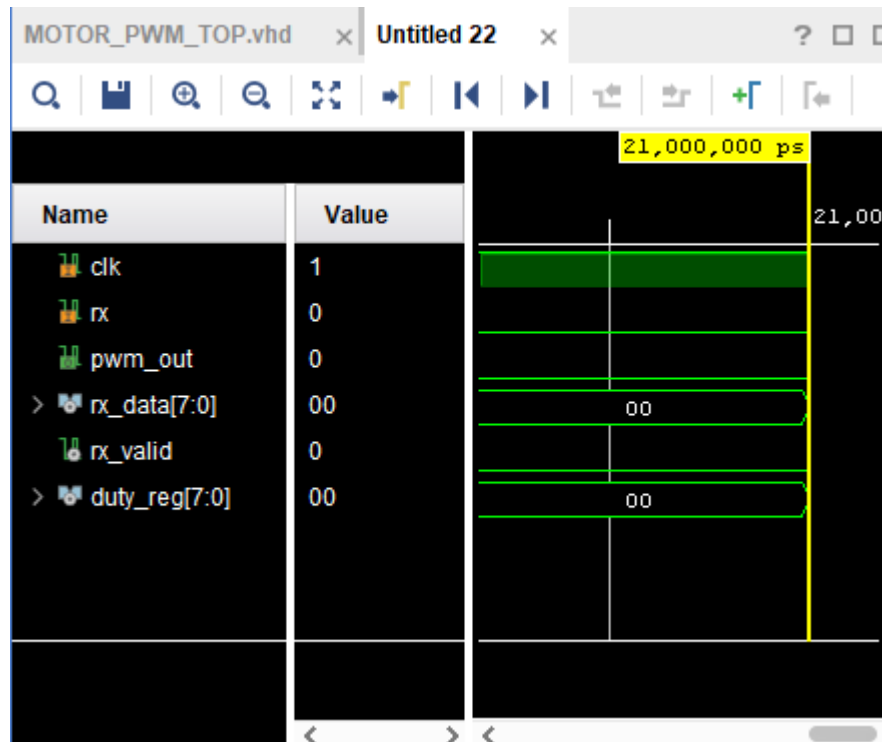
            duty_cnt <= (to_integer(unsigned(duty_perc)) * PWM_TOP) / 100;

            if pwm_cnt < duty_cnt then
                pwm_out <= '1';
            else
                pwm_out <= '0';
            end if;
        end if;
    end process;

end Behavioral;

```

Motor pwm



Name		Value	2,052,074,275,200 ps	
Name		Value	2,052,074,275,198 ps	2,052,074,275,202 ps
clk		1		
rx		1		
pwm_out		0		
> rx_data[7:0]		19	19	
rx_valid		1		
> duty_reg[7:0]		19	19	

Name		Value	2,667,948,505,598 ps	
Name		Value	2,667,948,505,596 ps	2,667,948,505,600 ps
clk		1		
rx		0		
pwm_out		1		
> rx_data[7:0]		19	19	
rx_valid		1		
> duty_reg[7:0]		19	19	


```

entity MOTOR_PWM_TOP is
Port ( clk : in STD_LOGIC;
      rx : in STD_LOGIC;
      pwm_out : out STD_LOGIC);
MOTOR_PWM_TOP;

architecture Behavioral of MOTOR_PWM_TOP is

component UART_RECEIVE is
    port (
        clock      : in  std_logic;
        serial_in   : in  std_logic;
        parallel_out : out std_logic_vector(7 downto 0);
        vallid      : out std_logic
    );
end component;

component PWM_GENERATOR is
    port (
        clk      : in  std_logic;
        duty_perc : in  std_logic_vector(7 downto 0);
        pwm_out   : out std_logic
    );
end component;

signal rx_data : std_logic_vector(7 downto 0);

```

```

signal rx_valid : std_logic;
signal duty_reg : std_logic_vector(7 downto 0) := (others => '0');

begin

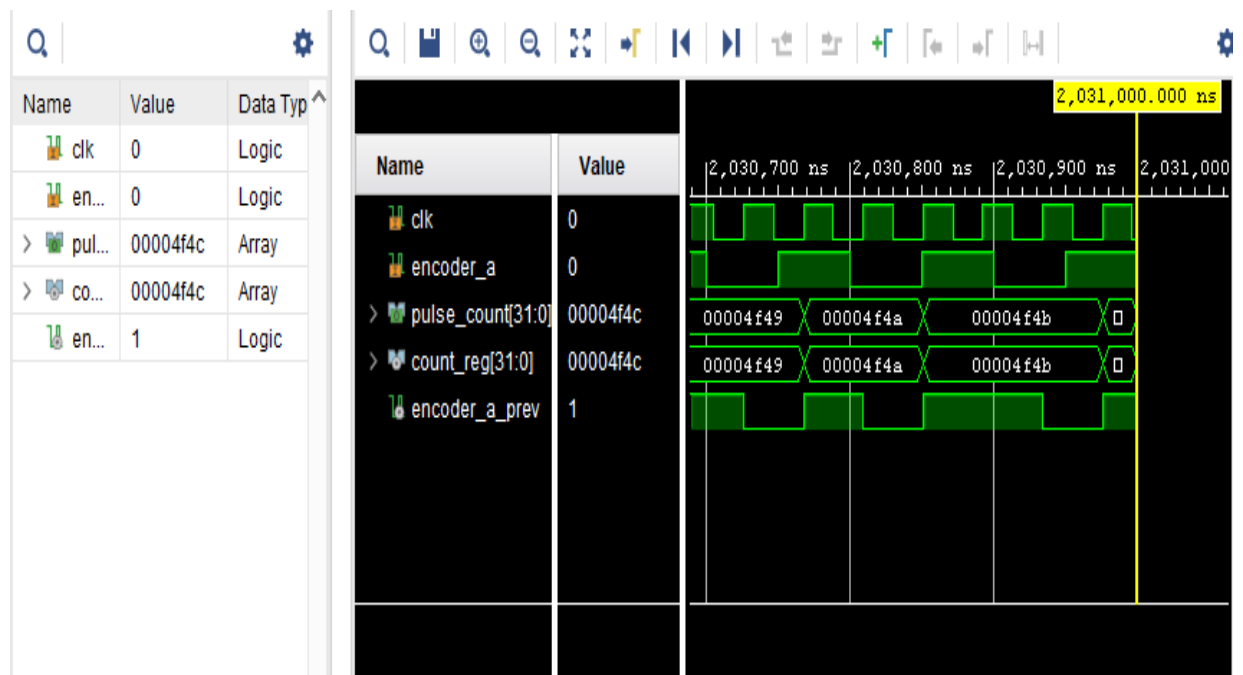
    U_RX: UART_RECEIVE
        port map (
            clock      => clk,
            serial_in   => rx,
            parallel_out => rx_data,
            vallid      => rx_valid
        );
    -- latch received percent
    process(clk)
    begin
        if rising_edge(clk) then
            if rx_valid = '1' then
                if unsigned(rx_data) > 100 then
                    duty_reg <= std_logic_vector(to_unsigned(100, 8));
                else
                    duty_reg <= rx_data;
                end if;
            end if;
        end if;
    end process;

    U_PWM: PWM_GENERATOR
        port map (
            clk      => clk,
            duty_perc => duty_reg,
            pwm_out   => pwm_out
        );

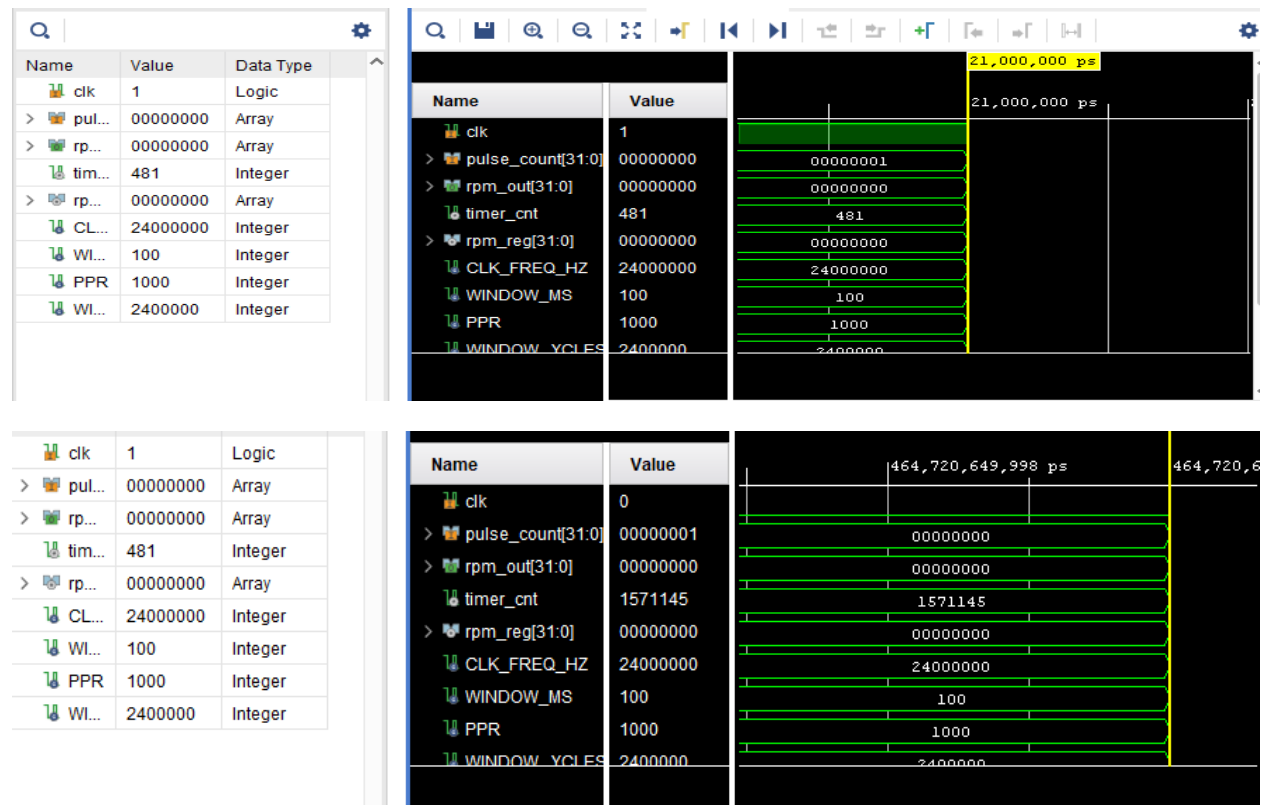
end Behavioral;

```

ENCODER:



PWM CONVERTOR:



Top modul encoder:

The screenshot displays a logic simulator interface with two main panes. The left pane, titled 'Objects', lists various signals and their current values and data types. The right pane shows a waveform view with a time scale of 361,000,000 ps. The 'uart_tx' signal is highlighted in blue.

Name	Value	Data Type
clk	1	Logic
en...	0	Logic
ua...	1	Logic
sta...	SEND_M	Enumera...
pul...	00000708	Array
rp...	000000de	Array
rp...	0	Logic
ua...	50	Array
ua...	0	Logic
ua...	1	Logic
rp...	000000de	Array
CL...	50000000	Integer
WI...	1000	Integer
PPR	20	Integer

Name	Value
clk	1
encoder_in	0
uart_tx	1
state	SEND_M
pulse_cnt[31:0]	00000708
rpm_sig[31:0]	000000de
rpm_valid	0
uart_data[7:0]	50
uart_start	0
uart_busy	1
rpm_work[31:0]	000000de
CLK_FREQ_HZ	50000000
WINDOW_MS	1000
PPR	20

The screenshot displays a logic simulator interface with two main panes. The left pane, titled 'Objects', lists various signals and their current values and data types. The right pane shows a waveform view with a time scale of 1,521,000,000 ps. The 'uart_tx' signal is highlighted in blue.

Name	Value	Data Type
clk	0	Logic
en...	0	Logic
ua...	0	Logic
sta...	SEND_N...	Enumera...
pul...	00001db0	Array
rp...	00001122	Array
rp...	0	Logic
ua...	3d	Array
ua...	0	Logic
ua...	1	Logic
rp...	000000de	Array
CL...	50000000	Integer
WI...	1000	Integer
PPR	20	Integer

Name	Value
clk	0
encoder_in	0
uart_tx	0
state	SEND_NUM
pulse_cnt[31:0]	00001db0
rpm_sig[31:0]	00001122
rpm_valid	0
uart_data[7:0]	3d
uart_start	0
uart_busy	1
rpm_work[31:0]	000000de
CLK_FREQ_HZ	50000000
WINDOW_MS	1000
PPR	20

```

entity EXERCISE5_TOP is
    generic (
        CLK_FREQ_HZ : positive := 50000000;
        WINDOW_MS   : positive := 1000;
        PPR          : positive := 20
    );
    port (
        clk       : in  std_logic;
        encoder_in : in  std_logic;
        uart_tx    : out std_logic
    );
end EXERCISE5_TOP;

architecture Behavioral of EXERCISE5_TOP is

    component ENCODER_COUNTER
        port (
            clk       : in  std_logic;
            encoder_a : in  std_logic;
            pulse_count : out unsigned(31 downto 0)
        );
    end component;

    component RPM_CONVERTOR
        generic (

```

```
        WINDOW_MS : positive := 1000;
        PPR        : positive := 20
    );
    port (
        clk          : in  std_logic;
        pulse_cnt    : in  unsigned(31 downto 0);
        rpm_out      : out unsigned(31 downto 0);
        rpm_valid    : out std_logic
    );
end component;

component UART_SEND
    port (
        clock        : in  std_logic;
        parallel_in   : in  std_logic_vector(7 downto 0);
        send          : in  std_logic;
        serial_out    : out std_logic;
        busy          : out std_logic
    );
end component;

type state_type is (
    IDLE,
    SEND_R,
    SEND_P,
    SEND_M,
```

```

        SEND_EQ,
        SEND_NUM,
        SEND_CR,
        SEND_LF
    );

    signal state      : state_type := IDLE;
    signal pulse_cnt  : unsigned(31 downto 0);
    signal rpm_sig    : unsigned(31 downto 0);
    signal rpm_valid  : std_logic;

    signal uart_data   : std_logic_vector(7 downto 0) := (others => '0');
    signal uart_start  : std_logic := '0';
    signal uart_busy   : std_logic;

    signal rpm_work    : unsigned(31 downto 0) := (others => '0');

begin

    U1: ENCODER_COUNTER
        port map (
            clk          => clk,
            encoder_a    => encoder_in,
            pulse_count  => pulse_cnt
        );

```

```

U2: RPM_CONVERTOR
    generic map (
        WINDOW_MS => WINDOW_MS,
        PPR        => PPR
    )
    port map (
        clk          => clk,
        pulse_cnt    => pulse_cnt,
        rpm_out      => rpm_sig,
        rpm_valid    => rpm_valid
    );

U3: UART_SEND
    port map (
        clock        => clk,
        parallel_in  => uart_data,
        send         => uart_start,
        serial_out   => uart_tx,
        busy         => uart_busy
    );

process(clk)
    variable digit : integer;
begin
    if rising_edge(clk) then

```

```
uart_start <= '0';

case state is
  when IDLE =>
    if rpm_valid = '1' then
      rpm_work <= rpm_sig;
      state    <= SEND_R;
    end if;

  when SEND_R =>
    if uart_busy = '0' then
      uart_data <= x"52"; -- 'R'
      uart_start <= '1';
      state     <= SEND_P;
    end if;

  when SEND_P =>
    if uart_busy = '0' then
      uart_data <= x"50"; -- 'P'
      uart_start <= '1';
      state     <= SEND_M;
    end if;

  when SEND_M =>
    if uart_busy = '0' then
      uart_data <= x"4D"; -- 'M'
```

```

        state      <= SEND_NUM;
    end if;
    when SEND_NUM =>
        if uart_busy = '0' then
            digit := to_integer(rpm_work mod 10);
            uart_data <= std_logic_vector(to_unsigned(character'pos('0') + digit, 8));
            uart_start <= '1';
            state      <= SEND_CR;
        end if;

    when SEND_CR =>
        if uart_busy = '0' then
            uart_data <= x"0D";
            uart_start <= '1';
            state      <= SEND_LF;
        end if;

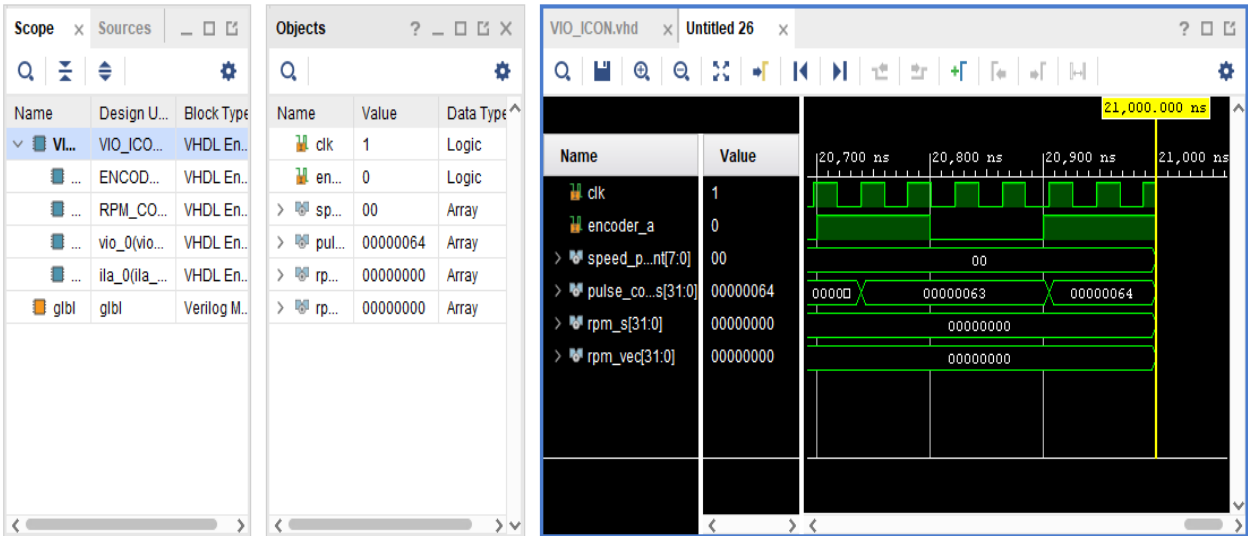
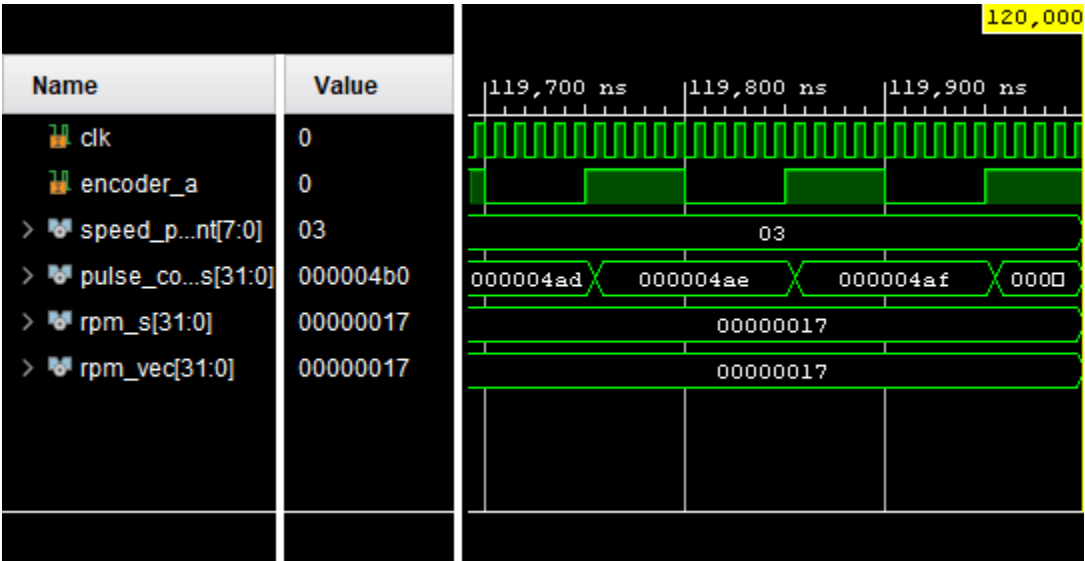
    when SEND_LF =>
        if uart_busy = '0' then
            uart_data <= x"0A";
            uart_start <= '1';
            state      <= IDLE;
        end if;
    end case;
end if;
end process;

        state      <= IDLE;
    end if;
end case;
end if;
end process;

end Behavioral;

```

VIO AND ICON(ILA);



```

entity VIO_ICON is
    port (
        clk          : in  std_logic;
        encoder_a     : in  std_logic
    );
end VIO_ICON;

architecture Behavioral of VIO_ICON is

    component ila_0
        port (
            clk      : in  std_logic;
            probe0    : in  std_logic_vector(7 downto 0);
            probe1    : in  std_logic_vector(31 downto 0);
            probe2    : in  std_logic_vector(31 downto 0)
        );
    end component;

    component vio_0
        port (
            clk      : in  std_logic;
            probe_in0 : in  std_logic_vector(31 downto 0);
            probe_out0 : out std_logic_vector(7 downto 0)
        );
    end component;

    component ENCODER_COUNTER
        port (
            clk      : in  std_logic;
            encoder_a : in  std_logic;
            pulse_count : out unsigned(31 downto 0)
        );
    end component;

    component RPM_CONVERTER
        generic (
            CLK_FREQ_HZ : integer := 24000000;
            WINDOW_MS   : integer := 100;
            PPR          : integer := 1000
        );
        port (
            clk      : in  std_logic;
            pulse_count : in  unsigned(31 downto 0);
            rpm_out   : out unsigned(31 downto 0)
        );
    end component;

```

```

signal speed_percent : std_logic_vector(7 downto 0);
signal pulse_count_s : unsigned(31 downto 0);
signal rpm_s         : unsigned(31 downto 0);
signal rpm_vec       : std_logic_vector(31 downto 0);

begin

    rpm_vec <= std_logic_vector(rpm_s);

    U_COUNTER : ENCODER_COUNTER
        port map (
            clk          => clk,
            encoder_a    => encoder_a,
            pulse_count => pulse_count_s
        );

    U_RPM : RPM_CONVERTER
        port map (
            clk          => clk,
            pulse_count => pulse_count_s,
            rpm_out     => rpm_s
        );

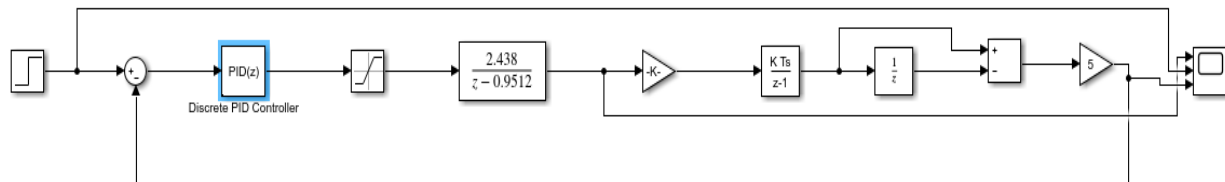
    U_VIO : vio_0
        port map (
            clk          => clk,
            probe_in0    => rpm_vec,
            probe_out0   => speed_percent
        );

    U_ILA : ila_0
        port map (
            clk          => clk,
            probe0       => speed_percent,
            probe1       => std_logic_vector(pulse_count_s),
            probe2       => rpm_vec
        );

end Behavioral;

```

Section 7: speed control system



Controller Parameters

	Tuned	Block
P	0.10856	0.1132
I	0.64264	0.56116
D	0.00047084	0
N	20.7488	100

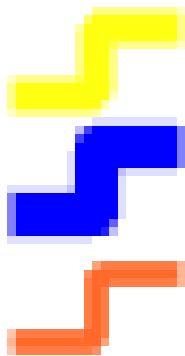
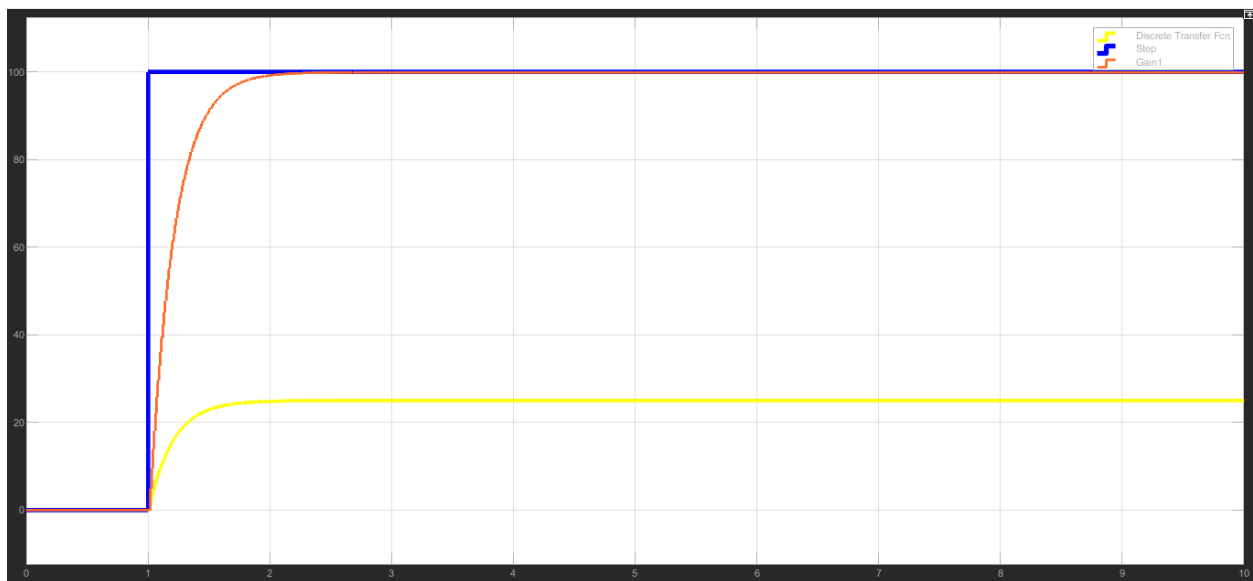
Performance and Robustness

	Tuned	Block
Rise time	0.05 seconds	0.05 seconds
Settling time	0.11 seconds	0.11 seconds
Overshoot	1.81 %	0.152 %
Peak	1.02	1
Gain margin	12.4 dB @ 105 rad/s	12.8 dB @ 104 rad/s
Phase margin	71 deg @ 23 rad/s	70 deg @ 23 rad/s
Closed-loop stability	Stable	Stable

```
Gc = tf(50, [0.2 1]);
Gd = c2d(Gc, 0.01, 'zoh');
```

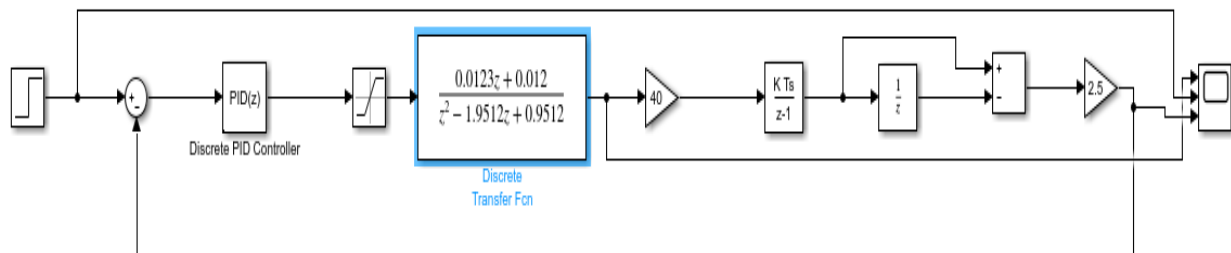
$$\frac{2.438}{z - 0.9512}$$

$$\frac{50}{0.2s + 1}$$



Discrete Transfer Fcn
Step
Gain1

Section 8: position control

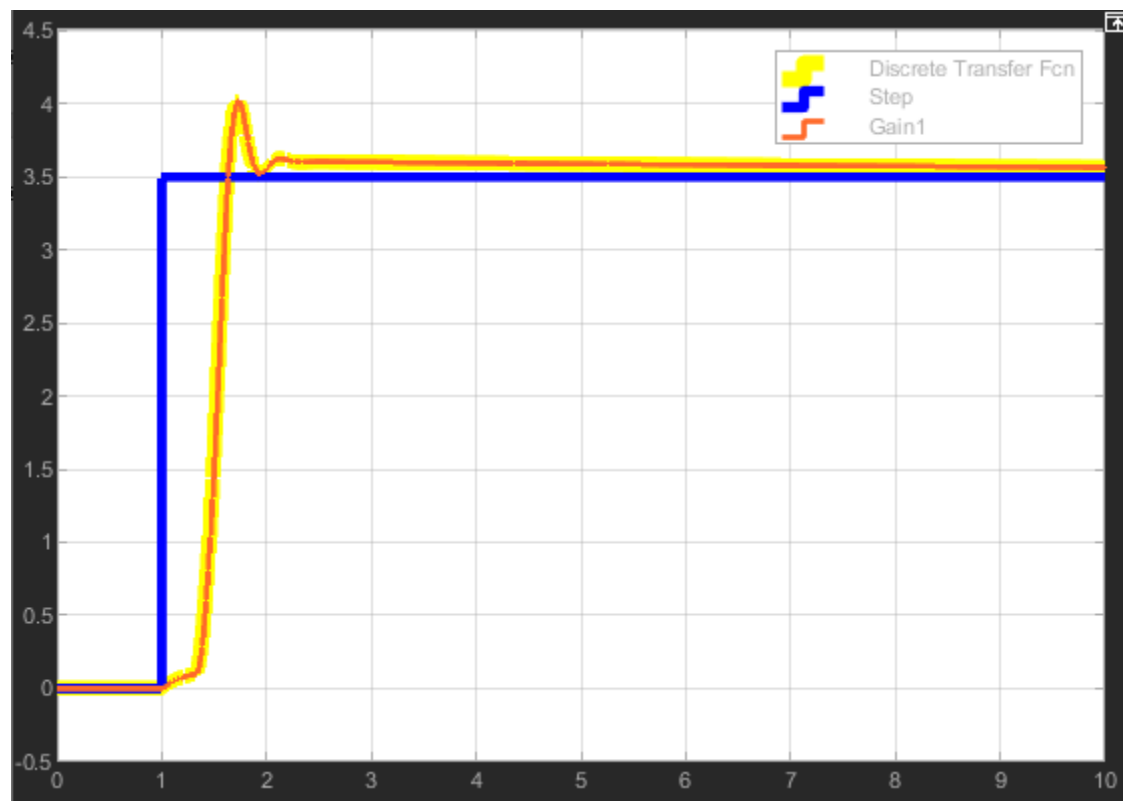


Controller Parameters

	Tuned
P	0.16896
I	0.062997
D	0.047399
N	193.03

Performance and Robustness

	Tuned
Rise time	0.01 seconds
Settling time	0.12 seconds
Overshoot	19.3 %
Peak	1.19
Gain margin	6.5 dB @ 105 rad/s
Phase margin	50.4 deg @ 47.6 rad/s
Closed-loop stability	Stable



```
Gc = tf(50,[0.2 1 0]);  
Gd = c2d(Gc,0.01,'zoh');
```

$$\frac{0.0123z + 0.012}{z^2 - 1.9512z + 0.9512}$$

$$\left(\frac{1}{s}\right)\left(\frac{50}{0.2s + 1}\right)$$

Section 8

```
clc;
clear;
close all;

pi_fixed = fi(pi,1,8,3);
pi_fixed.value

ans =

    '3.125'
```

به دلیل خطای کوانتیزاسیون مقدار واقعی با مقدار فعلی متفاوت است.

```
pi_fixed = fi(pi,1,8,3);
temp1 = pi_fixed.value;    %% ans = 3.125 ;
temp2 = pi_fixed.bin;      %% ans = 00011001 ;
```

c	h	temp1
c	h	temp2

'3.125'

'00011001'

```
clc;
clear;
close all;

pi_fixed      = fi(pi,1,8,3);
temp1         = pi_fixed.value; %% ans = '3.125' ;
temp2         = pi_fixed.bin;  %% ans = '00011001' ;

temp3         = str2num(temp1); %% ans = 3.125 ;
quantisation_error = pi - temp3; %% ans = 0.016 ;

quantisation_error =

    0.0166

>>
```

---(Floating-Point) ---

width_mm: 235 mm

aspect_ratio: 55%

rim_diameter_inch, rim_diameter_mm : 18.00 mm)

rim_diameter_inch, rim_diameter_mm : 457.20 mm)

height_mm :129.25 mm

wheel_diameter_mm: 715.70 mm

circumference_precise : 2248.4379 mm

```
% % % width_mm = 235;
% % % aspect_ratio = 55;
% % % rim_diameter_inch = 18;
% % % inch_to_mm = 25.4;
% % %
% % % %% CALCULATE height_mm:
% % % height_mm = (width_mm / 100) * aspect_ratio;
% % % %% CONVERT rim_diameter_INCH TO MM:
% % % rim_diameter_mm = rim_diameter_inch * inch_to_mm;
% % % %% CALCULATE wheel_diameter_mm:
% % % wheel_diameter_mm = rim_diameter_mm + 2 * height_mm;
% % % %% REAL PI
% % % circumference_precise = pi * wheel_diameter_mm;
% % %
% % % نمایش نتایج
% % % fprintf('---(Floating-Point) ---\n');
% % % fprintf('width_mm: %d mm\n', width_mm);
% % % fprintf('aspect_ratio: %d%%\n', aspect_ratio);
% % % fprintf('rim_diameter_inch, rim_diameter_mm : %.2f mm)\n', rim_diameter_inch, rim_diameter_mm);
% % % fprintf('height_mm :%.2f mm\n', height_mm);
% % % fprintf('wheel_diameter_mm: %.2f mm\n', wheel_diameter_mm);
```

FIXED POINT:

```
% Fixed-Point settings  
word_length = 16;  
fraction_length = 5;  
signed_mode = true;
```

Total wheel diameter (Fixed-Point): 677.3125 mm

Wheel circumference (Fixed-Point): 1023.9688 mm

Absolute error compared to exact calculation: 1224.469112 mm

```
% Fixed-Point settings  
word_length = 32;  
fraction_length = 10;  
signed_mode = true;
```

Total wheel diameter (Fixed-Point): 677.2070 mm

Wheel circumference (Fixed-Point): 2127.5146 mm

Absolute error compared to exact calculation: 120.923214 mm

```

% Parameters for tire
width_mm = 235;
aspect_ratio = 55;
rim_diameter_inch = 18;
inch_to_mm = 25.4;

% Fixed-Point settings
word_length = 32;
fraction_length = 9;
signed_mode = true;

% -- Fixed-Point Calculations --

% Convert constant and input values to fixed-point
width_fp = fi(width_mm, signed_mode, word_length, fraction_length);
aspect_ratio_fp = fi(aspect_ratio, signed_mode, word_length, fraction_length);
rim_diameter_inch_fp = fi(rim_diameter_inch, signed_mode, word_length, fraction_length);
inch_to_mm_fp = fi(inch_to_mm, signed_mode, word_length, fraction_length);
hundred_fp = fi(100, signed_mode, word_length, fraction_length);

two_fp = fi(2, signed_mode, word_length, fraction_length);
pi_fp = fi(pi, signed_mode, word_length, fraction_length);

% 1. Calculate sidewall height (H)
height_fp = fi((width_fp / hundred_fp) * aspect_ratio_fp, signed_mode, word_length, fraction_length);

% 2. Convert rim diameter to millimeters
rim_diameter_mm_fp = fi(rim_diameter_inch_fp * inch_to_mm_fp, signed_mode, word_length, fraction_length);

% 3. Calculate total wheel diameter (D_wheel)
multiplication_result_fp = fi(two_fp * height_fp, signed_mode, word_length, fraction_length);
wheel_diameter_fp = fi(rim_diameter_mm_fp + multiplication_result_fp, signed_mode, word_length, fraction_length);

wheel_diameter_fp_double = double(wheel_diameter_fp);
fprintf('Total wheel diameter (Fixed-Point): %.4f mm\n', wheel_diameter_fp_double);

% 4. Calculate wheel circumference
circumference_calc_fp = fi(pi_fp * wheel_diameter_fp, signed_mode, word_length, fraction_length);
circumference_fp = circumference_calc_fp; % Already quantized

circumference_fp_double = double(circumference_fp);

circumference_fp_double = double(circumference_fp);
fprintf('Wheel circumference (Fixed-Point): %.4f mm\n', circumference_fp_double);

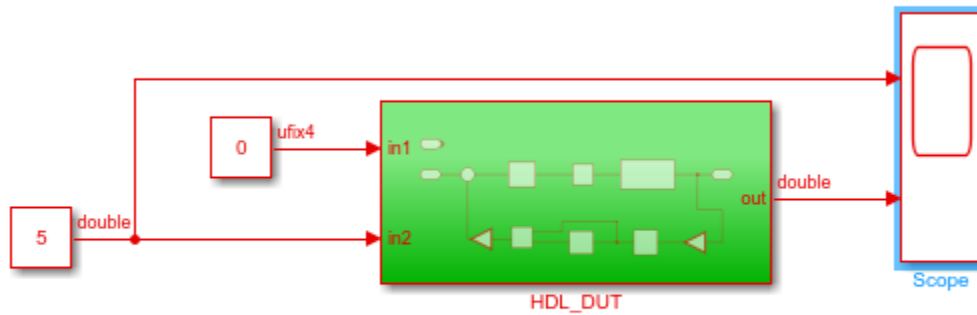
% Calculate and display error compared to exact value
width_mm_exact = 235;
aspect_ratio_exact = 55;
rim_diameter_inch_exact = 18;
height_mm_exact = (width_mm_exact / 100) * aspect_ratio_exact;
rim_diameter_mm_exact = rim_diameter_inch_exact * 25.4;
wheel_diameter_mm_exact = rim_diameter_mm_exact + 2 * height_mm_exact;
circumference_exact = pi * wheel_diameter_mm_exact;

error_val = abs(circumference_exact - circumference_fp_double);
fprintf('Absolute error compared to exact calculation: %.6f mm\n', error_val);

```

Section 10;

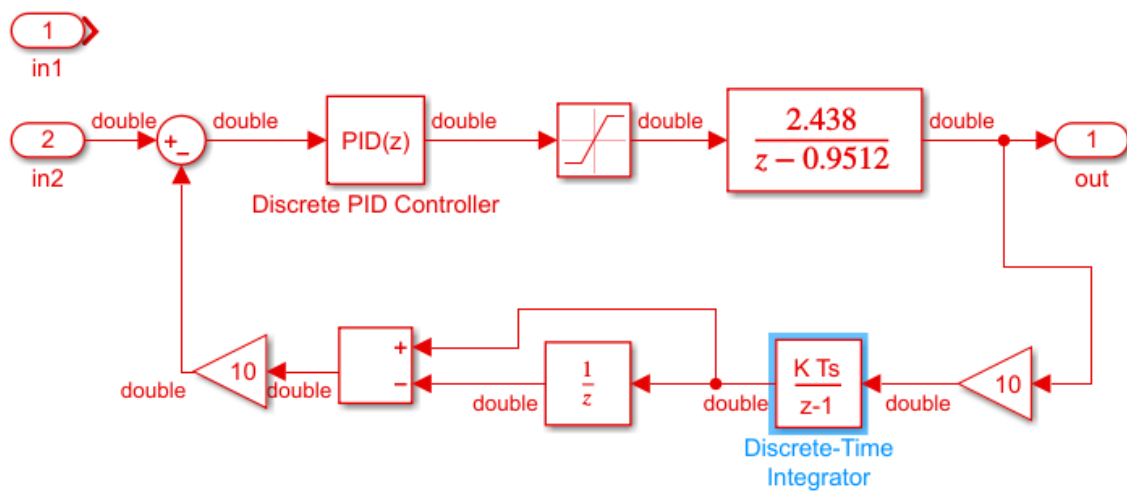
Note: This model is configured with 'hdlsetup'



Add your design targeted for ASIC/FPGA inside HDL_DUT and then run the following command:

```
makehdl("HDL_DUT")
```

Copyright 2015-2016 The MathWorks, Inc.



Contents

[Summary](#)

[Clock Summary](#)

[Code Interface Report](#)

[Timing And Area Report](#)

[High-level Resource Report](#)

[Critical Path Estimation](#)

[Optimization Report](#)

[Distributed Pipelining](#)

[Streaming and Sharing](#)

[Delay Balancing](#)

[Adaptive Pipelining](#)

[Traceability Report](#)

Generated Source Files

[Discrete_PID_Controller.vhd](#)

[HDL_DUT.vhd](#)

[untitled.vhd](#)

Referenced Models

HDL Code Generation Report Summary for untitled

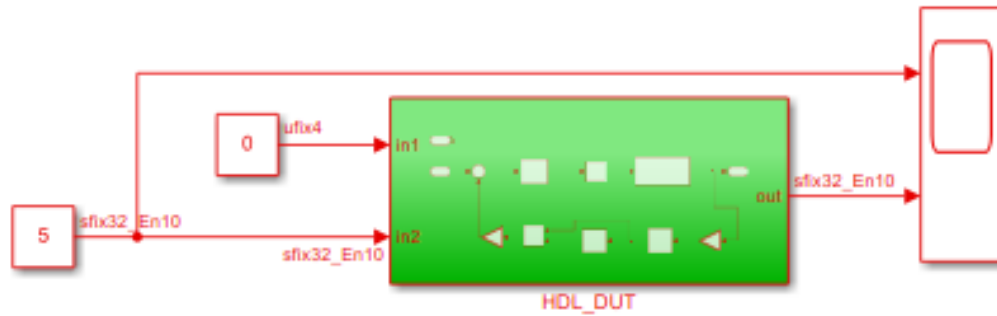
Summary

Model	untitled
Model version	1.0
HDL Coder version	3.13
HDL code generated on	2026-05-16 17:49:20
HDL code generated for	untitled
Target Language	VHDL
Target Directory	O:\your_desired_folder\Ali\zz\hdl_prj\hdlsrc

Non-default model properties

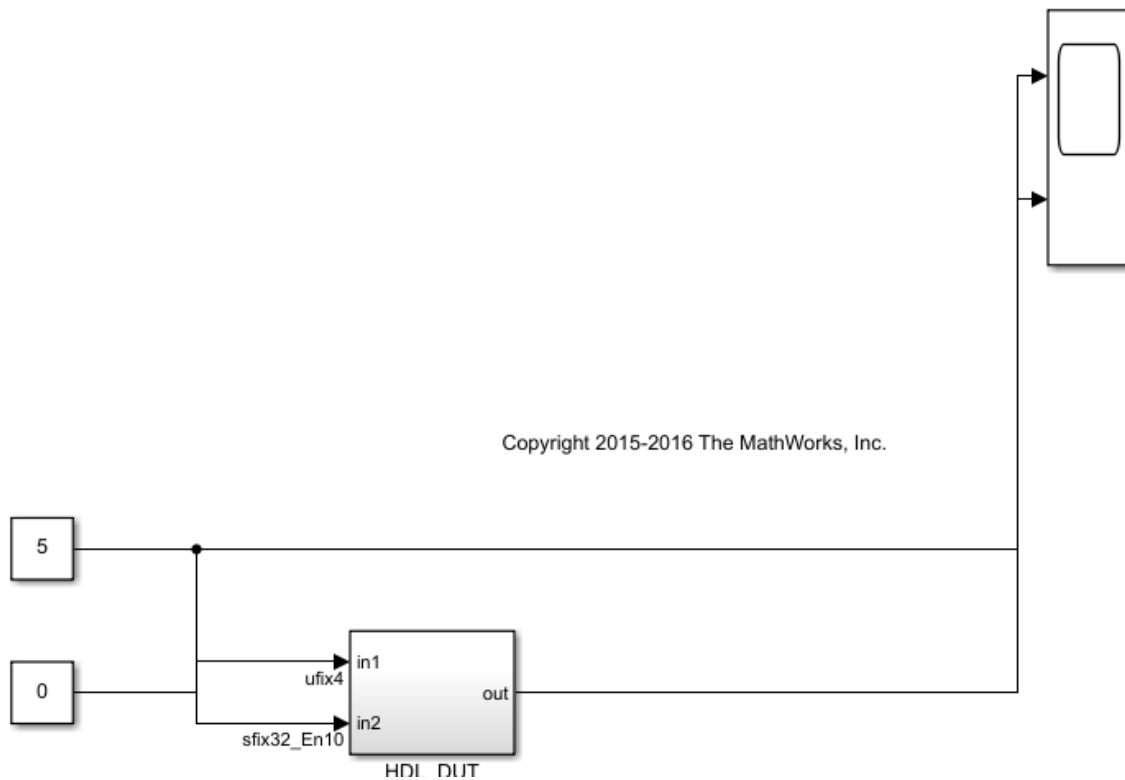
Backannotation	on
ClockHighTime	20
ClockLowTime	20
CriticalPathEstimation	on
GenerateHDLTestBench	off
GenerateValidationModel	on
HDLGenerateWebview	on
HDLSubsystem	untitled
OptimizationReport	on
ResourceReport	on
TargetDirectory	O:\your_desired_folder\Ali\zz\hdl_prj\hdlsrc
TargetFrequency	24
Traceability	on

Note: This model is configured with 'hdlsetup'



Add your design targeted for ASIC/FPGA inside HDL_DUT and then run the following command:
makehdl('HDL_DUT')

Copyright 2015-2016 The MathWorks, Inc.



Copyright 2015-2016 The MathWorks, Inc.

HDL_DUT_VHDL:

```
-----
18  LIBRARY IEEE;
19  USE IEEE.std_logic_1164.ALL;
20  USE IEEE.numeric_std.ALL;
21
22  ENTITY HDL_DUT IS
23      PORT( clk           : IN      std_logic;
24            reset         : IN      std_logic;
25            enb           : IN      std_logic;
26            in1           : IN      std_logic_vector(3 DOWNTO 0); -- ufix4
27            in2           : IN      std_logic_vector(31 DOWNTO 0); -- sfix32_En10
28            out_rsvd      : OUT     std_logic_vector(31 DOWNTO 0) -- sfix32_En10
29        );
30  END HDL_DUT;
31
32
33  ARCHITECTURE rtl OF HDL_DUT IS
34
35      -- Component Declarations
36      COMPONENT Discrete_PID_Controller
37      PORT( clk           : IN      std_logic;
38            reset         : IN      std_logic;
39            enb           : IN      std_logic;
40            u             : IN      std_logic_vector(31 DOWNTO 0); -- sfix32_En10
41            y             : OUT     std_logic_vector(31 DOWNTO 0) -- sfix32_En10
42        );
43  END COMPONENT;
44
45      -- Component Configuration Statements
46      FOR ALL : Discrete_PID_Controller
47          USE ENTITY work.Discrete_PID_Controller(rtl);
48
49      -- Signals
50      SIGNAL in2_signed      : signed(31 DOWNTO 0); -- sfix32_En10
51      SIGNAL s_state_out1    : signed(31 DOWNTO 0); -- sfix32_En10
52      SIGNAL denom_gain1_mul_temp : signed(63 DOWNTO 0); -- sfix64_En20
53      SIGNAL s_denom_gain1    : signed(31 DOWNTO 0); -- sfix32_En10
54      SIGNAL Discrete_Time_Integrator_x_reg : signed(31 DOWNTO 0); -- sfix32_En10
55      SIGNAL Unit_Delay_out1  : signed(31 DOWNTO 0); -- sfix32_En10
56      SIGNAL s_num gain1      : signed(31 DOWNTO 0); -- sfix32_En10
57      SIGNAL Gain1_mul_temp   : signed(63 DOWNTO 0); -- sfix64_En37
58      SIGNAL Gain1_out1       : signed(31 DOWNTO 0); -- sfix32_En10
59      SIGNAL Discrete_Time_Integrator_indtc : signed(31 DOWNTO 0); -- sfix32_En10
60      SIGNAL gain_mul_temp    : signed(63 DOWNTO 0); -- sfix64_En37
61      SIGNAL Discrete_Time_Integrator_u_gain : signed(31 DOWNTO 0); -- sfix32_En10
62      SIGNAL Discrete_Time_Integrator_u_dtc : signed(31 DOWNTO 0); -- sfix32_En10
63      SIGNAL Discrete_Time_Integrator_u_add : signed(31 DOWNTO 0); -- sfix32_En10
64      SIGNAL Subtract_out1    : signed(31 DOWNTO 0); -- sfix32_En10
65      SIGNAL Gain_mul_temp_1  : signed(63 DOWNTO 0); -- sfix64_En37
66      SIGNAL Gain_out1        : signed(31 DOWNTO 0); -- sfix32_En10
67      SIGNAL Sum_out1         : signed(31 DOWNTO 0); -- sfix32_En10
68      SIGNAL Discrete_PID_Controller_out1 : std_logic_vector(31 DOWNTO 0); -- ufix32
69      SIGNAL Discrete_PID_Controller_out1_signed : signed(31 DOWNTO 0); -- sfix32_En10
70      SIGNAL Saturation_out1  : signed(31 DOWNTO 0); -- sfix32_En10
71      SIGNAL s_denom_acc_out1 : signed(31 DOWNTO 0); -- sfix32_En10
72      SIGNAL num gain1_mul_temp : signed(63 DOWNTO 0); -- sfix64_En20
73
74  BEGIN
75      u_Discrete_PID_Controller : Discrete_PID_Controller
76      PORT MAP( clk => clk,
77                reset => reset,
78                enb => enb,
79                u => std_logic_vector(Sum_out1), -- sfix32_En10
80                y => Discrete_PID_Controller_out1 -- sfix32_En10
81        );
82
83      in2_signed <= signed(in2);
84
85      denom_gain1_mul_temp <= to_signed(-974, 32) * s_state_out1;
86      s_denom_gain1 <= denom_gain1_mul_temp(41 DOWNTO 10);
87
88      Unit_Delay_process : PROCESS (clk, reset)
89      BEGIN
90          IF reset = '1' THEN
91              Unit_Delay_out1 <= to_signed(0, 32);
92          ELSIF clk'EVENT AND clk = '1' THEN
93              IF enb = '1' THEN
```

```

94         Unit_Delay_out1 <= Discrete_Time_Integrator_x_reg;
95     END IF;
96 END IF;
97 END PROCESS Unit_Delay_process;
98
99
100 Gain1_mul_temp <= to_signed(1342177280, 32) * s_nume_gain1;
101 Gain1_out1 <= Gain1_mul_temp(58 DOWNT0 27);
102
103 Discrete_Time_Integrator_indtc <= Gain1_out1;
104
105 gain_mul_temp <= to_signed(1374389535, 32) * Discrete_Time_Integrator_indtc;
106 Discrete_Time_Integrator_u_gain <= resize(gain_mul_temp(63 DOWNT0 37), 32);
107
108 Discrete_Time_Integrator_u_dtc <= Discrete_Time_Integrator_u_gain;
109
110 Discrete_Time_Integrator_u_add <= Discrete_Time_Integrator_x_reg + Discrete_Time_Integrator_u_dtc;
111
112 Discrete_Time_Integrator_reg_process : PROCESS (clk, reset)
113 BEGIN
114     IF reset = '1' THEN
115         Discrete_Time_Integrator_x_reg <= to_signed(0, 32);
116     ELSIF clk'EVENT AND clk = '1' THEN
117         IF enb = '1' THEN
118             Discrete_Time_Integrator_x_reg <= Discrete_Time_Integrator_u_add;
119         END IF;
120     END IF;
121 END PROCESS Discrete_Time_Integrator_reg_process;
122
123
124 Subtract_out1 <= Discrete_Time_Integrator_x_reg - Unit_Delay_out1;
125
126 Gain_mul_temp_1 <= to_signed(1342177280, 32) * Subtract_out1;
127 Gain_out1 <= Gain_mul_temp_1(58 DOWNT0 27);
128
129 Sum_out1 <= in2_signed - Gain_out1;
130
131 Discrete_PID_Controller_out1_signed <= signed(Discrete_PID_Controller_out1);
132
133
134 Saturation_out1 <= to_signed(102400, 32) WHEN Discrete_PID_Controller_out1_signed > to_signed(102400,
32) ELSE
135     to_signed(0, 32) WHEN Discrete_PID_Controller_out1_signed < to_signed(0, 32) ELSE
136     Discrete_PID_Controller_out1_signed;
137
138 s_denom_acc_out1 <= Saturation_out1 - s_denom_gain1;
139
140 s_state_out1_1_process : PROCESS (clk, reset)
141 BEGIN
142     IF reset = '1' THEN
143         s_state_out1 <= to_signed(0, 32);
144     ELSIF clk'EVENT AND clk = '1' THEN
145         IF enb = '1' THEN
146             s_state_out1 <= s_denom_acc_out1;
147         END IF;
148     END IF;
149 END PROCESS s_state_out1_1_process;
150
151
152 nume_gain1_mul_temp <= to_signed(2497, 32) * s_state_out1;
153 s_nume_gain1 <= nume_gain1_mul_temp(41 DOWNT0 10);
154
155 out_rsvd <= std_logic_vector(s_nume_gain1);
156
157 END rtl;

```

DISCERIT_PID_VHDL_CODE:

```
-----
18  LIBRARY IEEE;
19  USE IEEE.std_logic_1164.ALL;
20  USE IEEE.numeric_std.ALL;
21
22  ENTITY Discrete_PID_Controller IS
23      PORT ( clk          : IN      std_logic;
24            reset         : IN      std_logic;
25            enb           : IN      std_logic;
26            u             : IN      std_logic_vector(31 DOWNTO 0); -- sfix32_En10
27            y             : OUT     std_logic_vector(31 DOWNTO 0) -- sfix32_En10
28          );
29  END Discrete_PID_Controller;
30
31
32  ARCHITECTURE rtl OF Discrete_PID_Controller IS
33
34      -- Signals
35      SIGNAL u_signed          : signed(31 DOWNTO 0); -- sfix32_En10
36      SIGNAL Proportional_Gain_mul_temp : signed(63 DOWNTO 0); -- sfix64_En20
37      SIGNAL Parallel_P_Gain_out1 : signed(31 DOWNTO 0); -- sfix32_En10
38      SIGNAL Integral_Gain_mul_temp : signed(63 DOWNTO 0); -- sfix64_En20
39      SIGNAL I_Gain_out1         : signed(31 DOWNTO 0); -- sfix32_En10
40      SIGNAL Integrator_indtc    : signed(31 DOWNTO 0); -- sfix32
41      SIGNAL gain_mul_temp       : signed(63 DOWNTO 0); -- sfix64_En37
42      SIGNAL Integrator_u_gain   : signed(31 DOWNTO 0); -- sfix32
43      SIGNAL Integrator_u_dtc    : signed(31 DOWNTO 0); -- sfix32_En10
44      SIGNAL Integrator_x_reg    : signed(31 DOWNTO 0); -- sfix32_En10
45      SIGNAL Integrator_u_add    : signed(31 DOWNTO 0); -- sfix32_En10
46      SIGNAL D_Gain_out1        : signed(31 DOWNTO 0); -- sfix32_En10
47      SIGNAL N_Gain_out1        : signed(31 DOWNTO 0); -- sfix32_En10
48      SIGNAL Filter_indtc       : signed(31 DOWNTO 0); -- sfix32
49      SIGNAL gain_mul_temp_1    : signed(63 DOWNTO 0); -- sfix64_En37
50      SIGNAL Filter_u_gain      : signed(31 DOWNTO 0); -- sfix32
51      SIGNAL Filter_u_dtc       : signed(31 DOWNTO 0); -- sfix32_En10
52      SIGNAL Filter_x_reg       : signed(31 DOWNTO 0); -- sfix32_En10
53      SIGNAL Filter_u_add       : signed(31 DOWNTO 0); -- sfix32_En10
54      SIGNAL Filter_out1        : signed(31 DOWNTO 0); -- sfix32_En10
55      SIGNAL Filter_Coefficient_mul_temp : signed(63 DOWNTO 0); -- sfix64_En20
56      SIGNAL Sum_add_temp       : signed(31 DOWNTO 0); -- sfix32_En10
57      SIGNAL Sum_out1          : signed(31 DOWNTO 0); -- sfix32_En10
58
59  BEGIN
60      -- Feedback Features
61      --
62      -- Derivative
63      --
64      -- Integrator
65
66      u_signed <= signed(u);
67
68      Proportional_Gain_mul_temp <= to_signed(102, 32) * u_signed;
69      Parallel_P_Gain_out1 <= Proportional_Gain_mul_temp(41 DOWNTO 10) + ('0' &
(Proportional_Gain_mul_temp(9) AND ((NOT Proportional_Gain_mul_temp(63)) OR (Proportional_Gain_mul_temp(8) OR
Proportional_Gain_mul_temp(7) OR Proportional_Gain_mul_temp(6) OR Proportional_Gain_mul_temp(5) OR
Proportional_Gain_mul_temp(4) OR Proportional_Gain_mul_temp(3) OR Proportional_Gain_mul_temp(2) OR
Proportional_Gain_mul_temp(1) OR Proportional_Gain_mul_temp(0)))));
70
71      Integral_Gain_mul_temp <= to_signed(658, 32) * u_signed;
72      I_Gain_out1 <= Integral_Gain_mul_temp(41 DOWNTO 10) + ('0' & (Integral_Gain_mul_temp(9) AND ((NOT
Integral_Gain_mul_temp(63)) OR (Integral_Gain_mul_temp(8) OR Integral_Gain_mul_temp(7) OR
Integral_Gain_mul_temp(6) OR Integral_Gain_mul_temp(5) OR Integral_Gain_mul_temp(4) OR
Integral_Gain_mul_temp(3) OR Integral_Gain_mul_temp(2) OR Integral_Gain_mul_temp(1) OR
Integral_Gain_mul_temp(0)))));
73
74      Integrator_indtc <= I_Gain_out1;
75
76      gain_mul_temp <= to_signed(1374389535, 32) * Integrator_indtc;
77      Integrator_u_gain <= (resize(gain_mul_temp(63 DOWNTO 37), 32)) + ('0' & (gain_mul_temp(36) AND ((NOT
gain_mul_temp(63)) OR (gain_mul_temp(35) OR gain_mul_temp(34) OR gain_mul_temp(33) OR gain_mul_temp(32) OR
gain_mul_temp(31) OR gain_mul_temp(30) OR gain_mul_temp(29) OR gain_mul_temp(28) OR gain_mul_temp(27) OR
gain_mul_temp(26) OR gain_mul_temp(25) OR gain_mul_temp(24) OR gain_mul_temp(23) OR gain_mul_temp(22) OR
gain_mul_temp(21) OR gain_mul_temp(20) OR gain_mul_temp(19) OR gain_mul_temp(18) OR gain_mul_temp(17) OR
gain_mul_temp(16) OR gain_mul_temp(15) OR gain_mul_temp(14) OR gain_mul_temp(13) OR gain_mul_temp(12) OR
gain_mul_temp(11) OR gain_mul_temp(10) OR gain_mul_temp(9) OR gain_mul_temp(8) OR gain_mul_temp(7) OR
gain_mul_temp(6) OR gain_mul_temp(5) OR gain_mul_temp(4) OR gain_mul_temp(3) OR gain_mul_temp(2) OR
gain_mul_temp(1) OR gain_mul_temp(0)))));
```

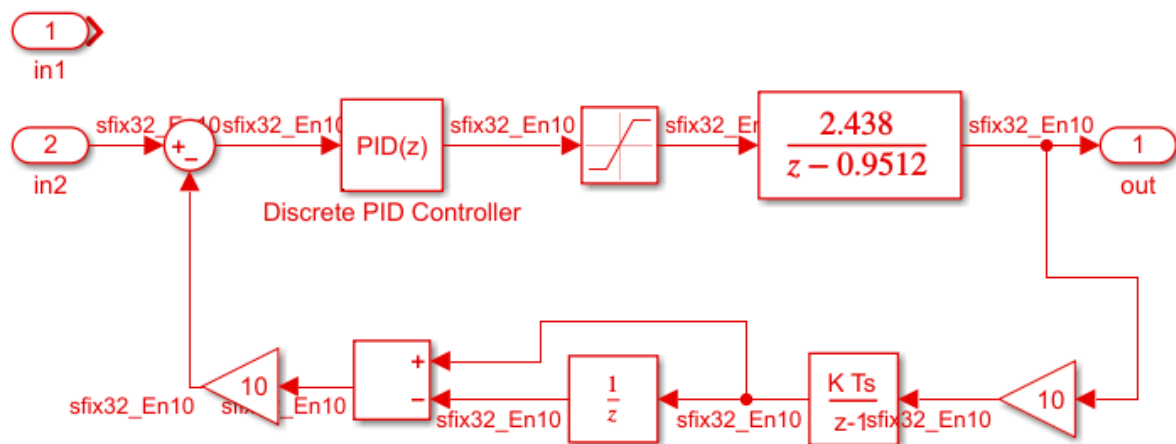
```

78
79   Integrator_u_dtc <= Integrator_u_gain;
80
81   Integrator_u_add <= Integrator_x_reg + Integrator_u_dtc;
82
83   Integrator_reg_process : PROCESS (clk, reset)
84   BEGIN
85       IF reset = '1' THEN
86           Integrator_x_reg <= to_signed(0, 32);
87       ELSIF clk'EVENT AND clk = '1' THEN
88           IF enb = '1' THEN
89               Integrator_x_reg <= Integrator_u_add;
90           END IF;
91       END IF;
92   END PROCESS Integrator_reg_process;
93
94
95   D_Gain_out1 <= to_signed(0, 32);
96
97   Filter_indtc <= N_Gain_out1;
98
99   gain_mul_temp_1 <= to_signed(1374389535, 32) * Filter_indtc;
100   Filter_u_gain <= (resize(gain_mul_temp_1(63 DOWNT0 37), 32)) + ('0' & (gain_mul_temp_1(36) AND (( NOT
gain_mul_temp_1(63)) OR (gain_mul_temp_1(35) OR gain_mul_temp_1(34) OR gain_mul_temp_1(33) OR
gain_mul_temp_1(32) OR gain_mul_temp_1(31) OR gain_mul_temp_1(30) OR gain_mul_temp_1(29) OR gain_mul_temp_1(28)
OR gain_mul_temp_1(27) OR gain_mul_temp_1(26) OR gain_mul_temp_1(25) OR gain_mul_temp_1(24) OR
gain_mul_temp_1(23) OR gain_mul_temp_1(22) OR gain_mul_temp_1(21) OR gain_mul_temp_1(20) OR gain_mul_temp_1(19)
OR gain_mul_temp_1(18) OR gain_mul_temp_1(17) OR gain_mul_temp_1(16) OR gain_mul_temp_1(15) OR
gain_mul_temp_1(14) OR gain_mul_temp_1(13) OR gain_mul_temp_1(12) OR gain_mul_temp_1(11) OR gain_mul_temp_1(10)
OR gain_mul_temp_1(9) OR gain_mul_temp_1(8) OR gain_mul_temp_1(7) OR gain_mul_temp_1(6) OR gain_mul_temp_1(5)
OR gain_mul_temp_1(4) OR gain_mul_temp_1(3) OR gain_mul_temp_1(2) OR gain_mul_temp_1(1) OR
gain_mul_temp_1(0))));
101
102   Filter_u_dtc <= Filter_u_gain;
103
104   Filter_u_add <= Filter_x_reg + Filter_u_dtc;
105
106   Filter_reg_process : PROCESS (clk, reset)
107   BEGIN
108       IF reset = '1' THEN
109           Filter_x_reg <= to_signed(0, 32);
110       ELSIF clk'EVENT AND clk = '1' THEN
111           IF enb = '1' THEN
112               Filter_x_reg <= Filter_u_add;
113           END IF;
114       END IF;
115   END PROCESS Filter_reg_process;
116
117
118   Filter_out1 <= D_Gain_out1 - Filter_x_reg;
119
120   Filter_Coefficient_mul_temp <= to_signed(21238, 32) * Filter_out1;
121   N_Gain_out1 <= Filter_Coefficient_mul_temp(41 DOWNT0 10) + ('0' & (Filter_Coefficient_mul_temp(9) AND
(( NOT Filter_Coefficient_mul_temp(63)) OR (Filter_Coefficient_mul_temp(8) OR Filter_Coefficient_mul_temp(7) OR
Filter_Coefficient_mul_temp(6) OR Filter_Coefficient_mul_temp(5) OR Filter_Coefficient_mul_temp(4) OR
Filter_Coefficient_mul_temp(3) OR Filter_Coefficient_mul_temp(2) OR Filter_Coefficient_mul_temp(1) OR
Filter_Coefficient_mul_temp(0))));
122
123   Sum_add_temp <= Parallel_P_Gain_out1 + Integrator_x_reg;
124   Sum_out1 <= Sum_add_temp + N_Gain_out1;
125
126   y <= std_logic_vector(Sum_out1);
127
128   END rtl;

```

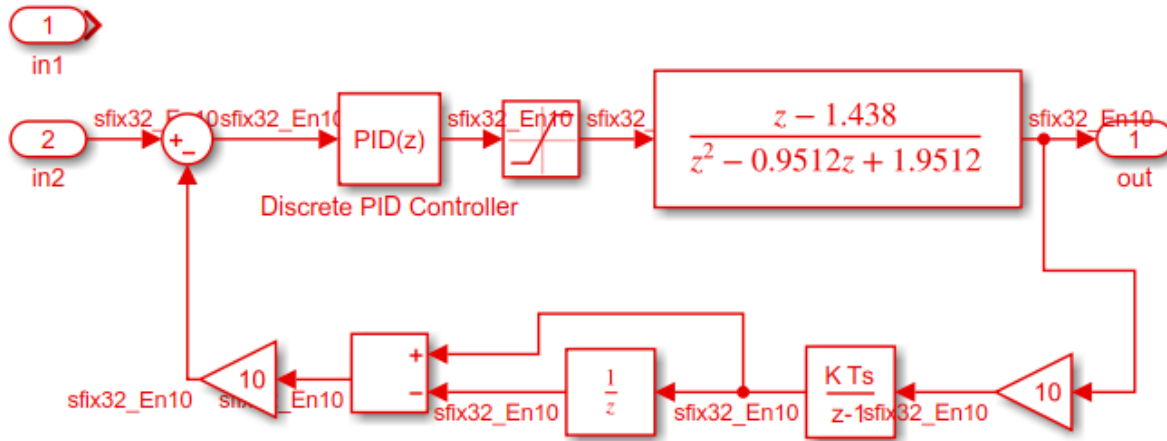
TOTAL_SYSTEM_CODE:

```
-----
25  LIBRARY IEEE;
26  USE IEEE.std_logic_1164.ALL;
27  USE IEEE.numeric_std.ALL;
28
29  ENTITY untitled IS
30      PORT( clk           : IN    std_logic;
31            reset         : IN    std_logic;
32            clk_enable    : IN    std_logic
33            );
34  END untitled;
35
36
37  ARCHITECTURE rtl OF untitled IS
38
39      -- Component Declarations
40      COMPONENT HDL_DUT
41          PORT( clk           : IN    std_logic;
42                reset        : IN    std_logic;
43                enb          : IN    std_logic;
44                in1          : IN    std_logic_vector(3 DOWNTO 0); --
ufix4
45                in2         : IN    std_logic_vector(31 DOWNTO 0); --
sfix32_En10
46                out_rsvd    : OUT   std_logic_vector(31 DOWNTO 0) --
sfix32_En10
47                );
48      END COMPONENT;
49
50      -- Component Configuration Statements
51      FOR ALL : HDL_DUT
52          USE ENTITY work.HDL_DUT(rtl);
53
54      -- Signals
55      SIGNAL Constant_out1      : unsigned(3 DOWNTO 0); -- ufix4
56      SIGNAL Constant1_out1     : signed(31 DOWNTO 0);  -- sfix32_En10
57      SIGNAL HDL_DUT_out1       : std_logic_vector(31 DOWNTO 0); -- ufix32
58
59      BEGIN
60          -- Note: This model is configured with 'hdlsetup'
61          --
62          -- Add your design targeted for ASIC/FPGA inside HDL_DUT and then run the following
command:
63          -- makehdl('HDL_DUT')
64          --
65          -- Copyright 2015-2016 The MathWorks, Inc.
66
67          u_HDL_DUT : HDL_DUT
68              PORT MAP( clk => clk,
69                      reset => reset,
70                      enb => clk_enable,
71                      in1 => std_logic_vector(Constant_out1), -- ufix4
72                      in2 => std_logic_vector(Constant1_out1), -- sfix32_En10
73                      out_rsvd => HDL_DUT_out1 -- sfix32_En10
74                      );
75
76
77          Constant_out1 <= to_unsigned(16#0#, 4);
78
79          Constant1_out1 <= to_signed(5120, 32);
80
81      END rtl;
```



SECTION10_2:

PID_DIGITAL_VHDL_CODE:



```

LIBRARY IEEE;
19  USE IEEE.std_logic_1164.ALL;
20  USE IEEE.numeric_std.ALL;
21
22  ENTITY Discrete_PID_Controller IS
23      PORT( clk
24             reset
25             enb
26             u
std_logic_vector(31 DOWNTO 0); -- sfix32_En10
27             y
std_logic_vector(31 DOWNTO 0) -- sfix32_En10
28             );
29  END Discrete_PID_Controller;
30
31
32  ARCHITECTURE rtl OF Discrete_PID_Controller IS
33
34      -- Signals
35      SIGNAL u_signed
sfix32_En10
36      SIGNAL Proportional_Gain_mul_temp
sfix64_En20
37      SIGNAL Parallel_P_Gain_out1
sfix32_En10
38      SIGNAL Integral_Gain_mul_temp
sfix64_En20
39      SIGNAL I_Gain_out1
sfix32_En10
40      SIGNAL Integrator_indtc
sfix32
41      SIGNAL gain_mul_temp
sfix64_En37
42      SIGNAL Integrator_u_gain
sfix32

```



```

43     SIGNAL Integrator_u_dtc                : signed(31 DOWNTO 0);  --
sfix32_En10
44     SIGNAL Integrator_x_reg                : signed(31 DOWNTO 0);  --
sfix32_En10
45     SIGNAL Integrator_u_add                : signed(31 DOWNTO 0);  --
sfix32_En10
46     SIGNAL D_Gain_out1                    : signed(31 DOWNTO 0);  --
sfix32_En10
47     SIGNAL N_Gain_out1                    : signed(31 DOWNTO 0);  --
sfix32_En10
48     SIGNAL Filter_indtc                   : signed(31 DOWNTO 0);  --
sfix32
49     SIGNAL gain_mul_temp_1                 : signed(63 DOWNTO 0);  --
sfix64_En37
50     SIGNAL Filter_u_gain                   : signed(31 DOWNTO 0);  --
sfix32
51     SIGNAL Filter_u_dtc                   : signed(31 DOWNTO 0);  --
sfix32_En10
52     SIGNAL Filter_x_reg                   : signed(31 DOWNTO 0);  --
sfix32_En10
53     SIGNAL Filter_u_add                   : signed(31 DOWNTO 0);  --
sfix32_En10
54     SIGNAL Filter_out1                    : signed(31 DOWNTO 0);  --
sfix32_En10
55     SIGNAL Filter_Coefficient_mul_temp     : signed(63 DOWNTO 0);  --
sfix64_En20
56     SIGNAL Sum_add_temp                   : signed(31 DOWNTO 0);  --
sfix32_En10
57     SIGNAL Sum_out1                       : signed(31 DOWNTO 0);  --
sfix32_En10
58
59     BEGIN
60         -- Feedback Features
61         --
62         -- Derivative
63         --
64         -- Integrator
65
66         u_signed <= signed(u);
67
68         Proportional_Gain_mul_temp <= to_signed(102, 32) * u_signed;
69         Parallel_P_Gain_out1 <= Proportional_Gain_mul_temp(41 DOWNTO 10) +
('0' & (Proportional_Gain_mul_temp(9) AND (( NOT
Proportional_Gain_mul_temp(63)) OR (Proportional_Gain_mul_temp(8) OR
Proportional_Gain_mul_temp(7) OR Proportional_Gain_mul_temp(6) OR
Proportional_Gain_mul_temp(5) OR Proportional_Gain_mul_temp(4) OR
Proportional_Gain_mul_temp(3) OR Proportional_Gain_mul_temp(2) OR
Proportional_Gain_mul_temp(1) OR Proportional_Gain_mul_temp(0)))));
70
71         Integral_Gain_mul_temp <= to_signed(658, 32) * u_signed;
72         I_Gain_out1 <= Integral_Gain_mul_temp(41 DOWNTO 10) + ('0' &
(Integral_Gain_mul_temp(9) AND (( NOT Integral_Gain_mul_temp(63)) OR
(Integral_Gain_mul_temp(8) OR Integral_Gain_mul_temp(7) OR
Integral_Gain_mul_temp(6) OR Integral_Gain_mul_temp(5) OR
Integral_Gain_mul_temp(4) OR Integral_Gain_mul_temp(3) OR
Integral_Gain_mul_temp(2) OR Integral_Gain_mul_temp(1) OR
Integral_Gain_mul_temp(0)))));

```

```

73
74     Integrator_indtc <= I_Gain_out1;
75
76     gain_mul_temp <= to_signed(1374389535, 32) * Integrator_indtc;
77     Integrator_u_gain <= (resize(gain_mul_temp(63 DOWNT0 37), 32)) +
('0' & (gain_mul_temp(36) AND ((NOT gain_mul_temp(63)) OR (gain_mul_temp(35)
OR gain_mul_temp(34) OR gain_mul_temp(33) OR gain_mul_temp(32) OR
gain_mul_temp(31) OR gain_mul_temp(30) OR gain_mul_temp(29) OR
gain_mul_temp(28) OR gain_mul_temp(27) OR gain_mul_temp(26) OR
gain_mul_temp(25) OR gain_mul_temp(24) OR gain_mul_temp(23) OR
gain_mul_temp(22) OR gain_mul_temp(21) OR gain_mul_temp(20) OR
gain_mul_temp(19) OR gain_mul_temp(18) OR gain_mul_temp(17) OR
gain_mul_temp(16) OR gain_mul_temp(15) OR gain_mul_temp(14) OR
gain_mul_temp(13) OR gain_mul_temp(12) OR gain_mul_temp(11) OR
gain_mul_temp(10) OR gain_mul_temp(9) OR gain_mul_temp(8) OR gain_mul_temp(7)
OR gain_mul_temp(6) OR gain_mul_temp(5) OR gain_mul_temp(4) OR
gain_mul_temp(3) OR gain_mul_temp(2) OR gain_mul_temp(1) OR
gain_mul_temp(0))));
78
79     Integrator_u_dtc <= Integrator_u_gain;
80
81     Integrator_u_add <= Integrator_x_reg + Integrator_u_dtc;
82
83     Integrator_reg_process : PROCESS (clk, reset)
84     BEGIN
85         IF reset = '1' THEN
86             Integrator_x_reg <= to_signed(0, 32);
87         ELSIF clk'EVENT AND clk = '1' THEN
88             IF enb = '1' THEN
89                 Integrator_x_reg <= Integrator_u_add;
90             END IF;
91         END IF;
92     END PROCESS Integrator_reg_process;
93
94
95     D_Gain_out1 <= to_signed(0, 32);
96
97     Filter_indtc <= N_Gain_out1;
98
99     gain_mul_temp_1 <= to_signed(1374389535, 32) * Filter_indtc;
100     Filter_u_gain <= (resize(gain_mul_temp_1(63 DOWNT0 37), 32)) + ('0'
& (gain_mul_temp_1(36) AND ((NOT gain_mul_temp_1(63)) OR
(gain_mul_temp_1(35) OR gain_mul_temp_1(34) OR gain_mul_temp_1(33) OR
gain_mul_temp_1(32) OR gain_mul_temp_1(31) OR gain_mul_temp_1(30) OR
gain_mul_temp_1(29) OR gain_mul_temp_1(28) OR gain_mul_temp_1(27) OR
gain_mul_temp_1(26) OR gain_mul_temp_1(25) OR gain_mul_temp_1(24) OR
gain_mul_temp_1(23) OR gain_mul_temp_1(22) OR gain_mul_temp_1(21) OR
gain_mul_temp_1(20) OR gain_mul_temp_1(19) OR gain_mul_temp_1(18) OR
gain_mul_temp_1(17) OR gain_mul_temp_1(16) OR gain_mul_temp_1(15) OR
gain_mul_temp_1(14) OR gain_mul_temp_1(13) OR gain_mul_temp_1(12) OR
gain_mul_temp_1(11) OR gain_mul_temp_1(10) OR gain_mul_temp_1(9) OR
gain_mul_temp_1(8) OR gain_mul_temp_1(7) OR gain_mul_temp_1(6) OR
gain_mul_temp_1(5) OR gain_mul_temp_1(4) OR gain_mul_temp_1(3) OR
gain_mul_temp_1(2) OR gain_mul_temp_1(1) OR gain_mul_temp_1(0))));
101
102     Filter_u_dtc <= Filter_u_gain;
103

```

```

104     Filter_u_add <= Filter_x_reg + Filter_u_dtc;
105
106     Filter_reg_process : PROCESS (clk, reset)
107     BEGIN
108         IF reset = '1' THEN
109             Filter_x_reg <= to_signed(0, 32);
110         ELSIF clk'EVENT AND clk = '1' THEN
111             IF enb = '1' THEN
112                 Filter_x_reg <= Filter_u_add;
113             END IF;
114         END IF;
115     END PROCESS Filter_reg_process;
116
117
118     Filter_out1 <= D_Gain_out1 - Filter_x_reg;
119
120     Filter_Coefficient_mul_temp <= to_signed(21238, 32) * Filter_out1;
121     N_Gain_out1 <= Filter_Coefficient_mul_temp(41 DOWNTO 10) + ('0' &
(Filter_Coefficient_mul_temp(9) AND (( NOT Filter_Coefficient_mul_temp(63))
OR (Filter_Coefficient_mul_temp(8) OR Filter_Coefficient_mul_temp(7) OR
Filter_Coefficient_mul_temp(6) OR Filter_Coefficient_mul_temp(5) OR
Filter_Coefficient_mul_temp(4) OR Filter_Coefficient_mul_temp(3) OR
Filter_Coefficient_mul_temp(2) OR Filter_Coefficient_mul_temp(1) OR
Filter_Coefficient_mul_temp(0))));
122
123     Sum_add_temp <= Parallel_P_Gain_out1 + Integrator_x_reg;
124     Sum_out1 <= Sum_add_temp + N_Gain_out1;
125
126     y <= std_logic_vector(Sum_out1);
127
128     END rtl;

```

```

LIBRARY IEEE;
19     USE IEEE.std_logic_1164.ALL;
20     USE IEEE.numeric_std.ALL;
21
22     ENTITY HDL_DUT IS
23     PORT( clk                : IN    std_logic;
24           reset              : IN    std_logic;
25           enb                : IN    std_logic;
26           in1                : IN    std_logic;
std_logic_vector(3 DOWNTO 0); -- ufix4
27           in2                : IN
std_logic_vector(31 DOWNTO 0); -- sfix32_En10
28           out_rsvd          : OUT
std_logic_vector(31 DOWNTO 0) -- sfix32_En10
29           );
30     END HDL_DUT;
31
32
33     ARCHITECTURE rtl OF HDL_DUT IS
34
35         -- Component Declarations
36         COMPONENT Discrete_PID_Controller
37         PORT( clk                : IN    std_logic;
38               reset              : IN    std_logic;
39               enb                : IN    std_logic;
40               u                  : IN
std_logic_vector(31 DOWNTO 0); -- sfix32_En10
41               y                  : OUT
std_logic_vector(31 DOWNTO 0) -- sfix32_En10
42               );
43         END COMPONENT;
44
45         -- Component Configuration Statements
46         FOR ALL : Discrete_PID_Controller
47             USE ENTITY work.Discrete_PID_Controller(rtl);
48
49         -- Signals
50         SIGNAL in2_signed        : signed(31 DOWNTO 0); --
sfix32_En10
51         SIGNAL s_state_out2      : signed(31 DOWNTO 0); --
sfix32_En10
52         SIGNAL nume_gain2_mul_temp : signed(63 DOWNTO 0); --
sfix64_En20
53         SIGNAL s_nume_gain2      : signed(31 DOWNTO 0); --
sfix32_En10
54         SIGNAL s_state_out1      : signed(31 DOWNTO 0); --
sfix32_En10
55         SIGNAL denom_gain2_mul_temp : signed(63 DOWNTO 0); --
sfix64_En20
56         SIGNAL s_denom_gain2     : signed(31 DOWNTO 0); --
sfix32_En10
57         SIGNAL denom_gain1_mul_temp : signed(63 DOWNTO 0); --
sfix64_En20

```

```

58     SIGNAL s_denom_gain1                : signed(31 DOWNT0 0);  --
sfix32_En10
59     SIGNAL Discrete_Time_Integrator_x_reg : signed(31 DOWNT0 0);  --
sfix32_En10
60     SIGNAL Unit_Delay_out1              : signed(31 DOWNT0 0);  --
sfix32_En10
61     SIGNAL s_num_acc_out1               : signed(31 DOWNT0 0);  --
sfix32_En10
62     SIGNAL Gain1_mul_temp               : signed(63 DOWNT0 0);  --
sfix64_En37
63     SIGNAL Gain1_out1                   : signed(31 DOWNT0 0);  --
sfix32_En10
64     SIGNAL Discrete_Time_Integrator_indtc : signed(31 DOWNT0 0);  --
sfix32
65     SIGNAL gain_mul_temp                : signed(63 DOWNT0 0);  --
sfix64_En37
66     SIGNAL Discrete_Time_Integrator_u_gain : signed(31 DOWNT0 0);  --
sfix32
67     SIGNAL Discrete_Time_Integrator_u_dtc : signed(31 DOWNT0 0);  --
sfix32_En10
68     SIGNAL Discrete_Time_Integrator_u_add : signed(31 DOWNT0 0);  --
sfix32_En10
69     SIGNAL Subtract_out1                : signed(31 DOWNT0 0);  --
sfix32_En10
70     SIGNAL Gain_mul_temp_1              : signed(63 DOWNT0 0);  --
sfix64_En37
71     SIGNAL Gain_out1                    : signed(31 DOWNT0 0);  --
sfix32_En10
72     SIGNAL Sum_out1                     : signed(31 DOWNT0 0);  --
sfix32_En10
73     SIGNAL Discrete_PID_Controller_out1  : std_logic_vector(31
DOWNT0 0);  -- ufix32
74     SIGNAL Discrete_PID_Controller_out1_signed : signed(31 DOWNT0 0);
-- sfix32_En10
75     SIGNAL Saturation_out1              : signed(31 DOWNT0 0);  --
sfix32_En10
76     SIGNAL s_denom_acc_out1             : signed(31 DOWNT0 0);  --
sfix32_En10
77     SIGNAL s_denom_acc_out2             : signed(31 DOWNT0 0);  --
sfix32_En10
78
79     BEGIN
80         u_Discrete_PID_Controller : Discrete_PID_Controller
81         PORT MAP( clk => clk,
82                  reset => reset,
83                  enb => enb,
84                  u => std_logic_vector(Sum_out1),  -- sfix32_En10
85                  y => Discrete_PID_Controller_out1  -- sfix32_En10
86                );
87
88         in2_signed <= signed(in2);
89
90         nume_gain2_mul_temp <= to_signed(-1473, 32) * s_state_out2;
91         s_num_gain2 <= nume_gain2_mul_temp(41 DOWNT0 10);
92
93         s_state_out2_1_process : PROCESS (clk, reset)
94         BEGIN

```

```

95     IF reset = '1' THEN
96         s_state_out2 <= to_signed(0, 32);
97     ELSIF clk'EVENT AND clk = '1' THEN
98         IF enb = '1' THEN
99             s_state_out2 <= s_state_out1;
100         END IF;
101     END IF;
102 END PROCESS s_state_out2_1_process;
103
104
105     denom_gain2_mul_temp <= to_signed(1998, 32) * s_state_out2;
106     s_denom_gain2 <= denom_gain2_mul_temp(41 DOWNTO 10);
107
108     denom_gain1_mul_temp <= to_signed(-974, 32) * s_state_out1;
109     s_denom_gain1 <= denom_gain1_mul_temp(41 DOWNTO 10);
110
111 Unit_Delay_process : PROCESS (clk, reset)
112 BEGIN
113     IF reset = '1' THEN
114         Unit_Delay_out1 <= to_signed(0, 32);
115     ELSIF clk'EVENT AND clk = '1' THEN
116         IF enb = '1' THEN
117             Unit_Delay_out1 <= Discrete_Time_Integrator_x_reg;
118         END IF;
119     END IF;
120 END PROCESS Unit_Delay_process;
121
122
123     Gain1_mul_temp <= to_signed(1342177280, 32) * s_nume_acc_out1;
124     Gain1_out1 <= Gain1_mul_temp(58 DOWNTO 27);
125
126     Discrete_Time_Integrator_indtc <= Gain1_out1;
127
128     gain_mul_temp <= to_signed(1374389535, 32) *
Discrete_Time_Integrator_indtc;
129     Discrete_Time_Integrator_u_gain <= resize(gain_mul_temp(63 DOWNTO
37), 32);
130
131     Discrete_Time_Integrator_u_dtc <= Discrete_Time_Integrator_u_gain;
132
133     Discrete_Time_Integrator_u_add <= Discrete_Time_Integrator_x_reg +
Discrete_Time_Integrator_u_dtc;
134
135 Discrete_Time_Integrator_reg_process : PROCESS (clk, reset)
136 BEGIN
137     IF reset = '1' THEN
138         Discrete_Time_Integrator_x_reg <= to_signed(0, 32);
139     ELSIF clk'EVENT AND clk = '1' THEN
140         IF enb = '1' THEN
141             Discrete_Time_Integrator_x_reg <=
Discrete_Time_Integrator_u_add;
142         END IF;
143     END IF;
144 END PROCESS Discrete_Time_Integrator_reg_process;
145
146
147     Subtract_out1 <= Discrete_Time_Integrator_x_reg - Unit_Delay_out1;

```

```

148
149 Gain_mul_temp_1 <= to_signed(1342177280, 32) * Subtract_out1;
150 Gain_out1 <= Gain_mul_temp_1(58 DOWNT0 27);
151
152 Sum_out1 <= in2_signed - Gain_out1;
153
154 Discrete_PID_Controller_out1_signed <=
signed(Discrete_PID_Controller_out1);
155
156
157 Saturation_out1 <= to_signed(102400, 32) WHEN
Discrete_PID_Controller_out1_signed > to_signed(102400, 32) ELSE
158 to_signed(0, 32) WHEN Discrete_PID_Controller_out1_signed <
to_signed(0, 32) ELSE
159 Discrete_PID_Controller_out1_signed;
160
161 s_denom_acc_out1 <= Saturation_out1 - s_denom_gain1;
162
163 s_denom_acc_out2 <= s_denom_acc_out1 - s_denom_gain2;
164
165 s_state_out1_1_process : PROCESS (clk, reset)
166 BEGIN
167 IF reset = '1' THEN
168 s_state_out1 <= to_signed(0, 32);
169 ELSIF clk'EVENT AND clk = '1' THEN
170 IF enb = '1' THEN
171 s_state_out1 <= s_denom_acc_out2;
172 END IF;
173 END IF;
174 END PROCESS s_state_out1_1_process;
175
176
177 s_nume_acc_out1 <= s_state_out1 + s_nume_gain2;
178
179 out_rsvd <= std_logic_vector(s_nume_acc_out1);
180
181 END rtl;

```

SECTION10_2_VHDL_CODE:

```
LIBRARY IEEE;
26  USE IEEE.std_logic_1164.ALL;
27  USE IEEE.numeric_std.ALL;
28
29  ENTITY SECTION10_2 IS
30      PORT( clk
31              reset
32              clk_enable
33              );
34  END SECTION10_2;
35
36
37  ARCHITECTURE rtl OF SECTION10_2 IS
38
39      -- Component Declarations
40      COMPONENT HDL_DUT
41          PORT( clk
42                  reset
43                  enb
44                  in1
std_logic_vector(3 DOWNTO 0); -- ufix4
45                  in2
std_logic_vector(31 DOWNTO 0); -- sfix32_En10
46                  out_rsvd
std_logic_vector(31 DOWNTO 0) -- sfix32_En10
47                  );
48      END COMPONENT;
49
50      -- Component Configuration Statements
51      FOR ALL : HDL_DUT
52          USE ENTITY work.HDL_DUT(rtl);
53
54      -- Signals
55      SIGNAL Constant_out1
ufix4
56      SIGNAL Constant1_out1
sfix32_En10
57      SIGNAL HDL_DUT_out1
DOWNTO 0); -- ufix32
58
59      BEGIN
60          -- Note: This model is configured with 'hdlsetup'
61          --
62          -- Add your design targeted for ASIC/FPGA inside HDL_DUT and then
run the following command:
63          -- makehdl('HDL_DUT')
64          --
65          -- Copyright 2015-2016 The MathWorks, Inc.
66
67          u_HDL_DUT : HDL_DUT
68              PORT MAP( clk => clk,
69                      reset => reset,
70                      enb => clk_enable,
71                      in1 => std_logic_vector(Constant_out1), -- ufix4
```


Code Generation Report

Find:

Match Case

Contents

- Summary
- Code Interface Report
- Timing And Area Report
- High-level Resource Report
- Critical Path Estimation
- Optimization Report
- Distributed Pipelining
- Streaming and Sharing
- Delay Balancing
- Adaptive Pipelining
- Traceability Report

Generated Source Files

- Discrete_PID_Controller.vhd
- HDL_DUT.vhd
- SECTION10_2.vhd

Referenced Models

```

57 BEGIN
58   -- Note: This model is configured with 'hdlsetup'
59
60   -- Add your design targeted for ASIC/FPGA inside HDL_DUT and then run the following command:
61   -- makehdl('HDL_DUT')
62
63   -- Copyright 2015-2016 The MathWorks, Inc.
64
65   u_HDL_DUT : HDL_DUT
66     PORT MAP ( clk => clk,
67               reset => reset,
68               enb => clk_enable,
69               in1 => sfx32_logic_vector(Constant1_out1), -- sfix4
70               in2 => sfx32_logic_vector(Constant1_out1), -- sfix32_En10
71               out_read => HDL_DUT_out1 -- sfix32_En10
72             );
73
74   Constant1_out1 <= to_unsigned(10486, 4);
75   Constant1_out1 <= to_signed(5120, 32);
76
77   END;
78
79
80
81
82

```

HDL_DUT | View All

SECTION10_2 | HDL_DUT

Discrete PID Controller

HDL_DUT

Main

ShowPortLabels	FromPortIcon
Permissions	ReadWrite
ErrorFcn	
PermitHierarchical...	All
TreatAsAtomicUnit	off
TreatAsGroupedW...	on

Code Generation

RTWSystemCode	Auto
Other	

OK Help

در این بخش بنده از تولید کننده کد VHDL از برنامه سیمولینک متلب استفاده نمودم که به شدت جهت یادگیری این بخش از متلب مفید بود.

با تشکر